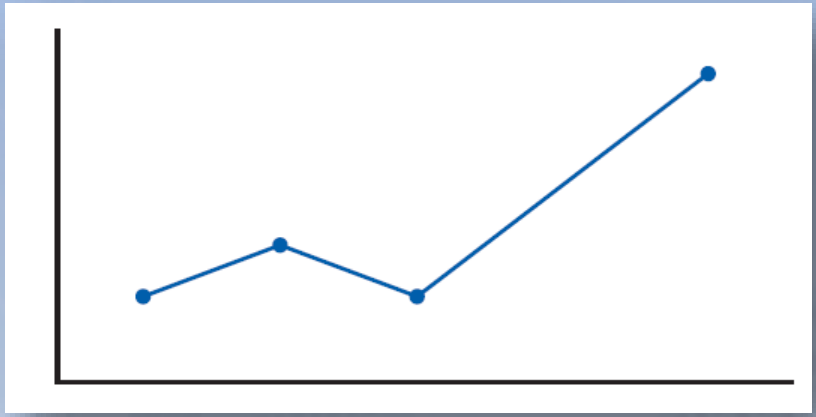
Graphs

Chapter 20

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology

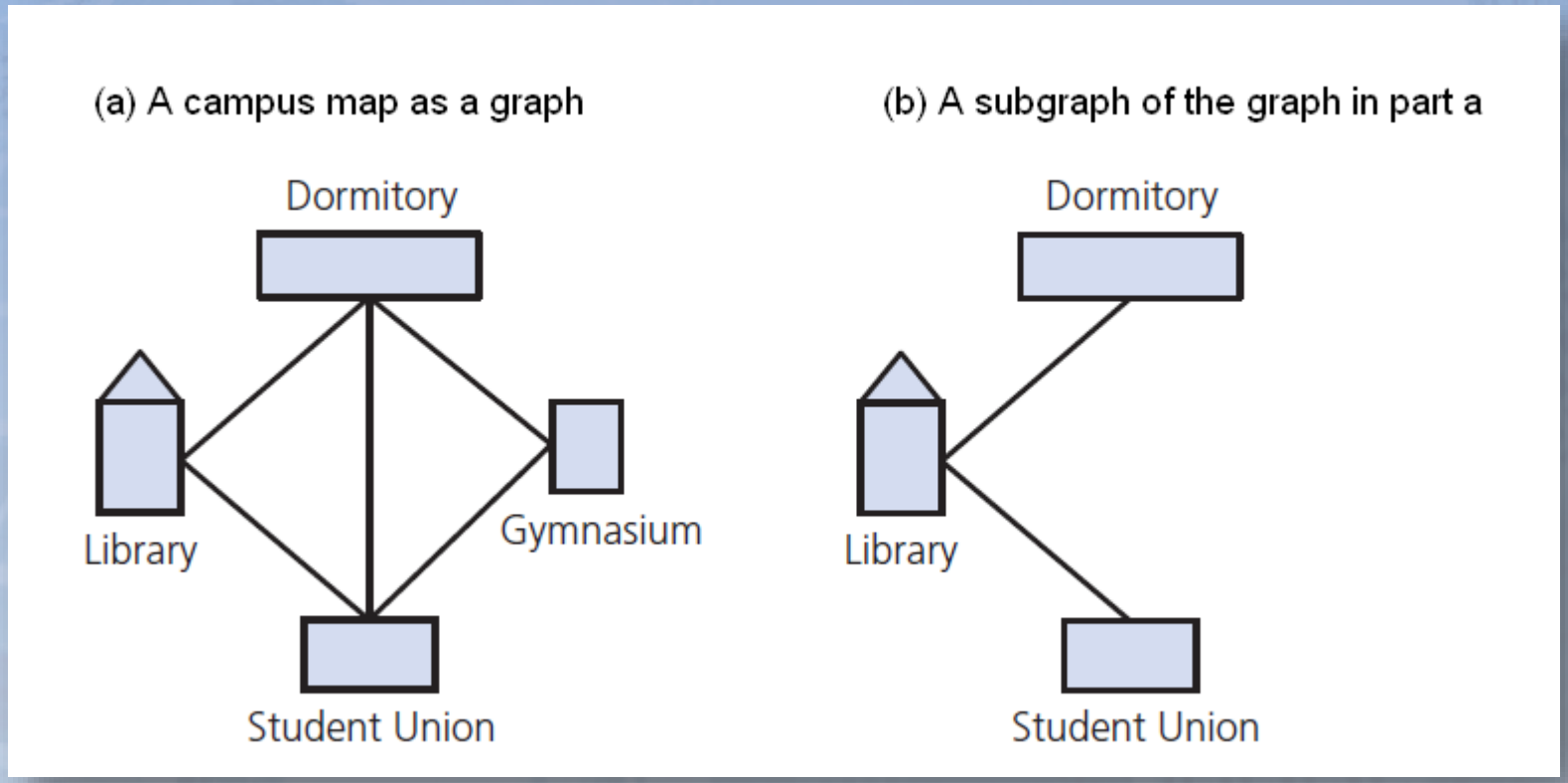
FIGURE 20-1
An ordinary
line graph



- In the context of this book, graphs represent relations among data items
- G = { V, E}
  - A graph is a set of vertices (nodes) and
  - A set of edges that connect the vertices

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology

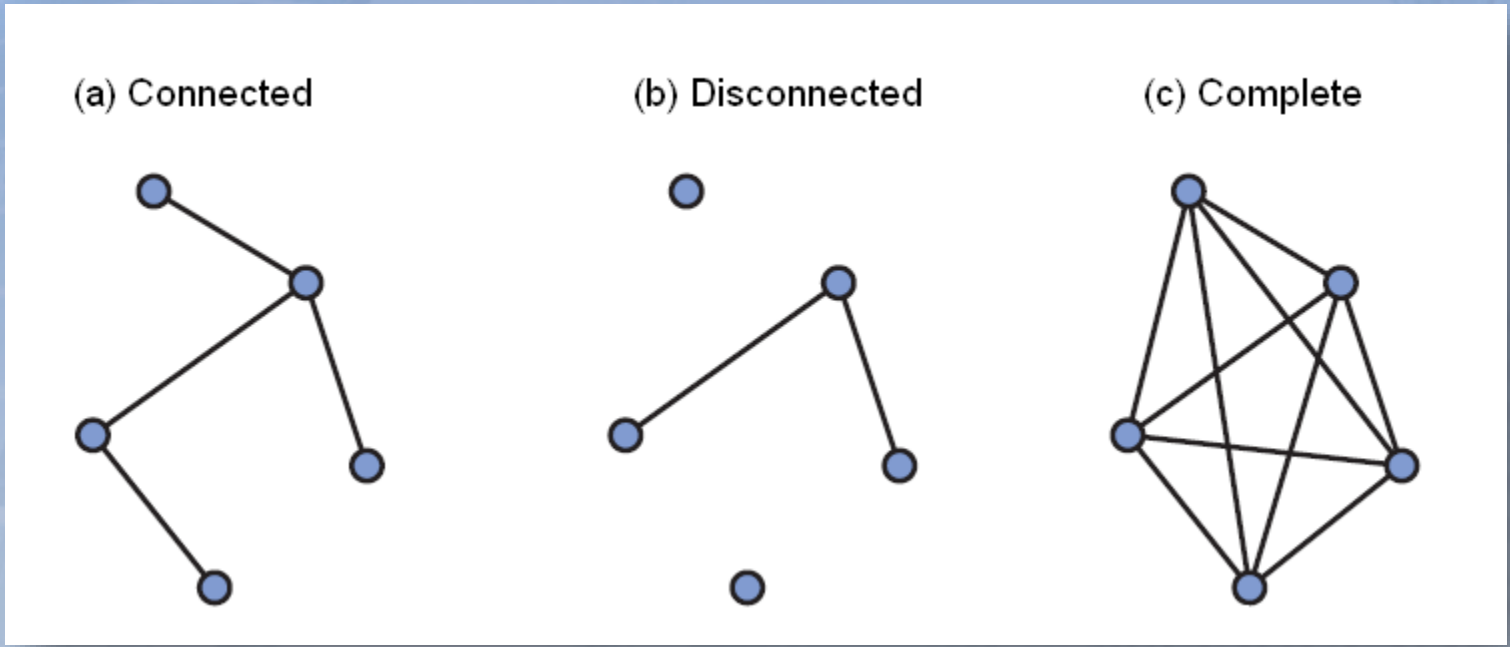


- Two vertices of a graph are **adjacent** if they are joined by an edge
- A **path** between two vertices is a sequence of edges that begins at one vertex and ends at another vertex

FIGURE 20-2 A graph and one of its subgraphs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology

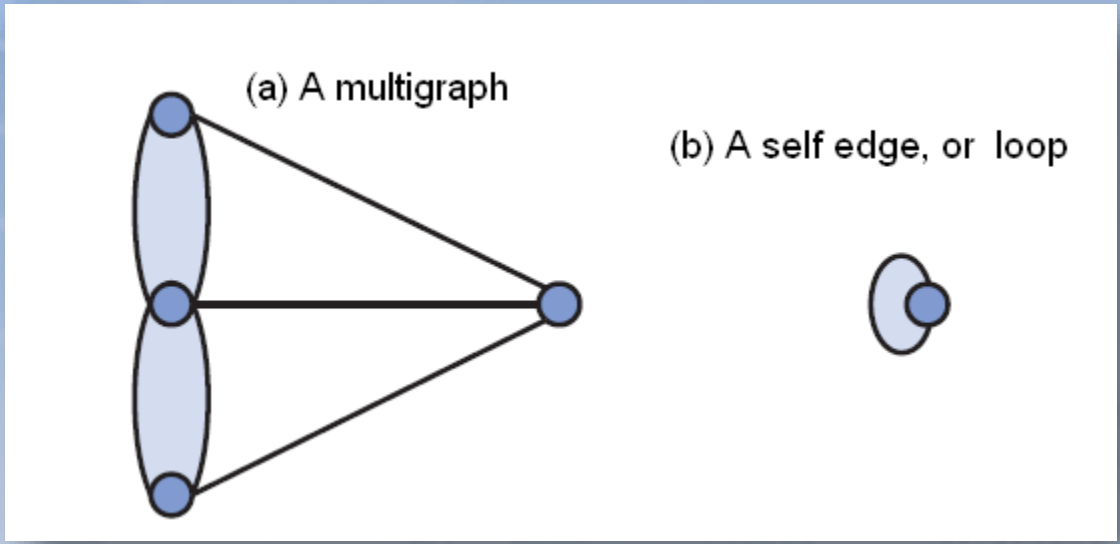


- A graph is **connected** if each pair of distinct vertices has a path between them
- In a **complete** graph, each pair of distinct vertices has an edge between them
- A **cycle** is a path that begins and ends at the same vertex
- A **simple cycle** is a cycle that does not pass through other vertices more than once

FIGURE 20-3 Examples of graphs that are either connected, disconnected, or complete

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology

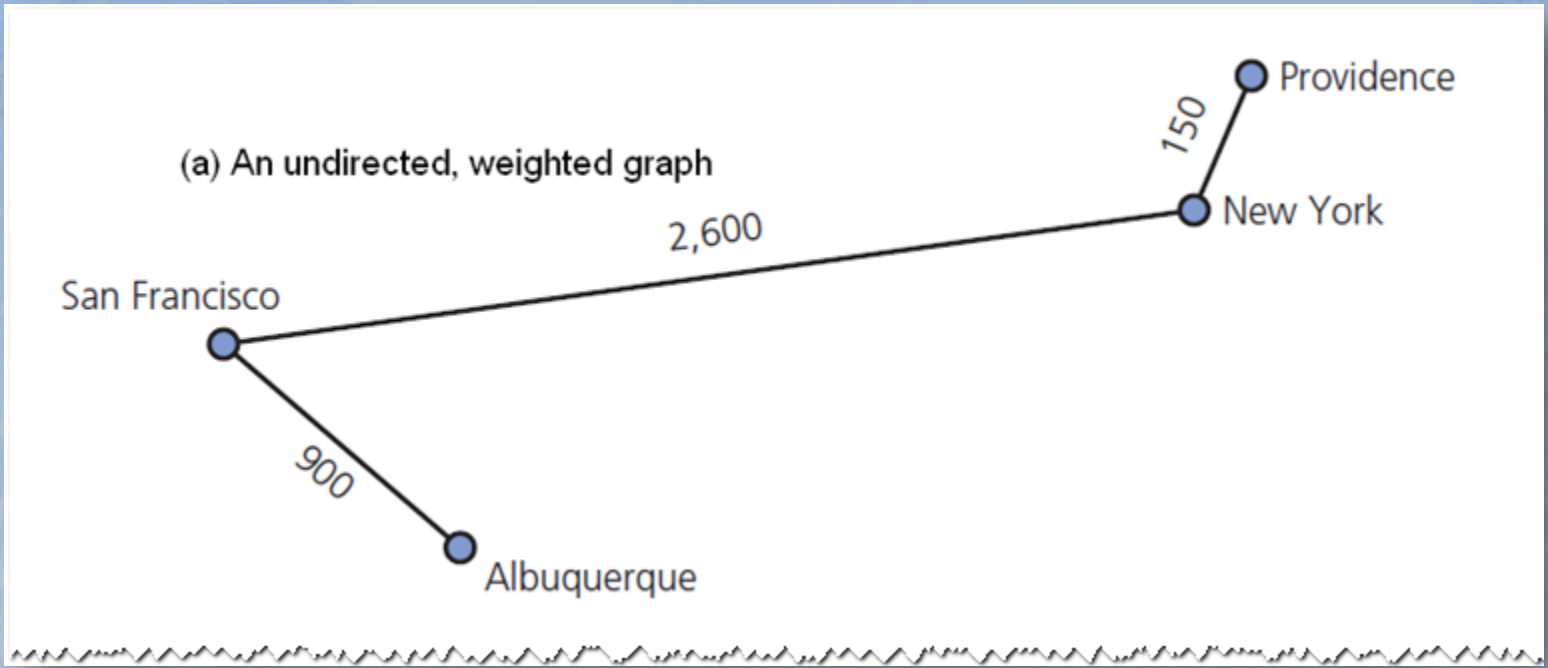


- Because a graph has a **set** of edges, a graph cannot have duplicate edges between vertices
- A multigraph does allow multiple edges; thus, a multigraph is **not** a graph
- A graph's edges cannot begin and end at the same vertex, a.k.a. self edge or loop

FIGURE 20-4 Graph-like structures that are not graphs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology

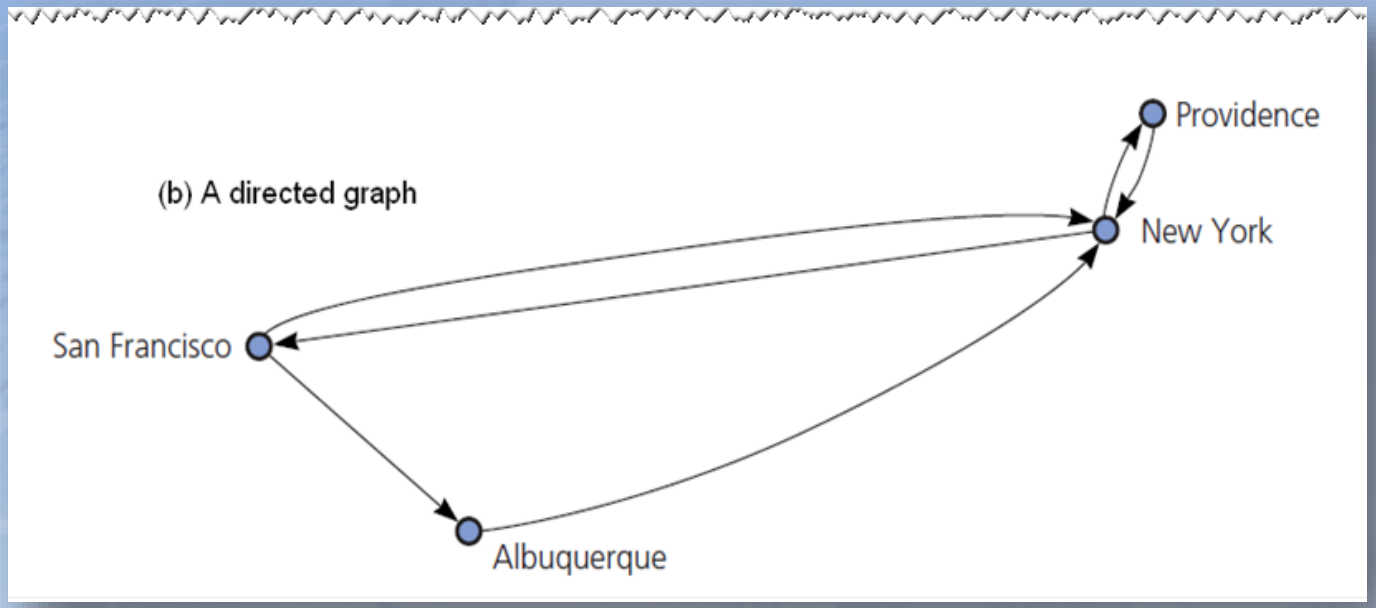


A **weighted** graph has a set of edges with **numeric** values.

FIGURE 20-5 (a) A weighted graph

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology



- What is the indegree of New York?
  - 3
- What is the outdegree of San Francisco?
  - 2

- Each edge in a directed graph, or digraph, has a **direction** and is called a directed edge
- A directed graph can have two edges between a pair of vertices, one in each direction
- A directed **path** is a sequence of directed edges between two vertices, e.g., there is a directed path that begins in Providence, goes to New York, and ends in San Francisco
- If there is a directed edge from vertex x to vertex y, then y is **adjacent** to x.
  - Albuquerque is adjacent to San Francisco, but San Francisco is not adjacent to Albuquerque
- Indegree – number of edges going into a node
- Outdegree – number of edges coming out of a node

FIGURE 20-5 Examples of two kinds of graphs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Graphs as ADTs

ADT graph operations

- Test if empty
- Get number of vertices, edges in a graph
- See if edge exists between two given vertices
- Add vertex to graph whose vertices have distinct, different values from new vertex
- Add/remove edge between two given vertices
- Remove vertex, edges to other vertices
- Retrieve vertex that contains given value
- Traverse the graph

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Graphs as ADTs

```
/** An interface for the ADT undirected, connected graph.
 @file GraphInterface.h */
#ifndef GRAPH_INTERFACE_
#define GRAPH_INTERFACE_

template<class LabelType>
class GraphInterface
{
public:
   /** Gets the number of vertices in this graph.
    @return  The number of vertices in the graph. */
   virtual int getNumVertices() const = 0;

   /** Gets the number of edges in this graph.
    @return  The number of edges in the graph. */
   virtual int getNumEdges() const = 0;
```

LISTING 20-1 A C++ interface for undirected, connected graphs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Graphs as ADTs

```
    /** Creates an undirected edge in this graph between two vertices
        that have the given labels. If such vertices do not exist, creates
        them and adds them to the graph before creating the edge.
     @param start  A label for the first vertex.
     @param end  A label for the second vertex.
     @param edgeWeight  The integer weight of the edge.
     @return  True if the edge is created, or false otherwise. */
    virtual bool add(LabelType start, LabelType end, int edgeWeight) = 0;

    /** Removes an edge from this graph. If a vertex is left with no other edges,
        it is removed from the graph since this is a connected graph.
     @param start  A label for the vertex at the beginning of the edge.
     @param end  A label for the vertex at the end of the edge.
     @return  True if the edge is removed, or false otherwise. */
    virtual bool remove(LabelType start, LabelType end) = 0;

    /** Gets the weight of an edge in this graph.
     @return  The weight of the specified edge.
        If no such edge exists, returns a negative integer. */
    virtual int getEdgeWeight(LabelType start, LabelType end) const = 0;
```

LISTING 20-1 A C++ interface for undirected, connected graphs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

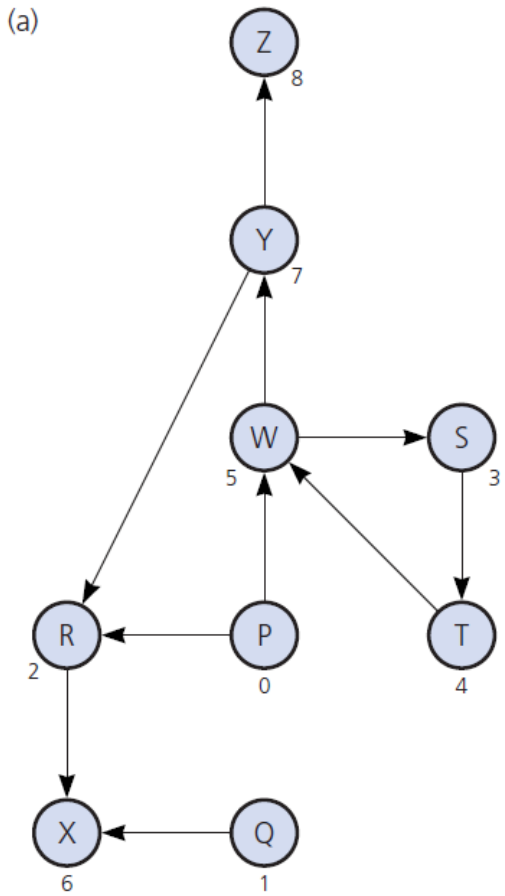
Graphs as ADTs

```
    /** Performs a depth-first search of this graph beginning at the given
        vertex and calls a given function once for each vertex visited.
     @param start  A label for the beginning vertex.
     @param visit  A client-defined function that performs an operation on
        or with each visited vertex. */
    virtual void depthFirstTraversal(LabelType start, void visit(LabelType&)) = 0;

    /** Performs a breadth-first search of this graph beginning at the given
        vertex and calls a given function once for each vertex visited.
     @param start  A label for the beginning vertex.
     @param visit  A client-defined function that performs an operation on
        or with each visited vertex. */
    virtual void breadthFirstTraversal(LabelType start, void visit(LabelType&)) = 0;

    /** Destroys this graph and frees its assigned memory. */
    virtual ~GraphInterface() {   }
};  // end GraphInterface
#endif
```

LISTING 20-1 A C++ interface for undirected, connected graphs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Implementing Graphs



(b)

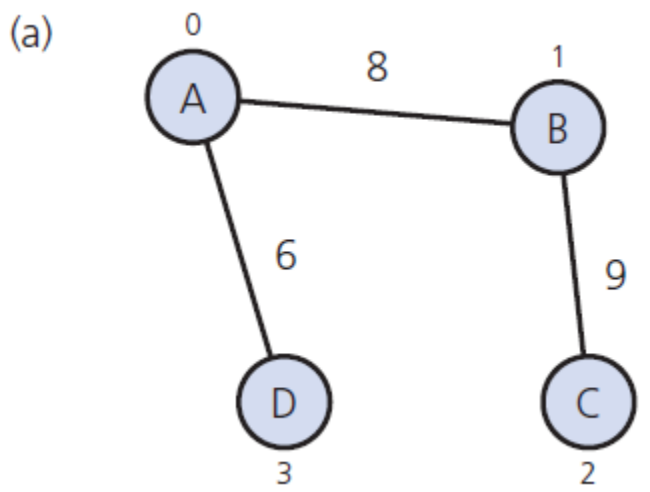
		0	1	2	3	4	5	6	7	8
		P	Q	R	S	T	W	X	Y	Z
0	P	0	0	1	0	0	1	0	0	0
1	Q	0	0	0	0	0	0	1	0	0
2	R	0	0	0	0	0	0	1	0	0
3	S	0	0	0	0	1	0	0	0	0
4	T	0	0	0	0	0	1	0	0	0
5	W	0	0	0	1	0	0	0	1	0
6	X	0	0	0	0	0	0	0	0	0
7	Y	0	0	1	0	0	0	0	0	1
8	Z	0	0	0	0	0	0	0	0	0

An adjacency matrix for a graph with n vertices numbered 0, 1, . . ., n – 1 is an n by n array matrix such that matrix[i][j] is 1 (true) if there is an edge from vertex i to vertex j, and 0 (false) otherwise.

FIGURE 20-7 (a) A directed graph and (b) its adjacency matrix

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Implementing Graphs



(b)

		0 A	1 B	2 C	3 D
0	A	∞	8	∞	6
1	B	8	∞	9	∞
2	C	∞	9	∞	∞
3	D	6	∞	∞	∞

FIGURE 20-8 (a) A weighted undirected graph and (b) its adjacency matrix

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Implementing Graphs

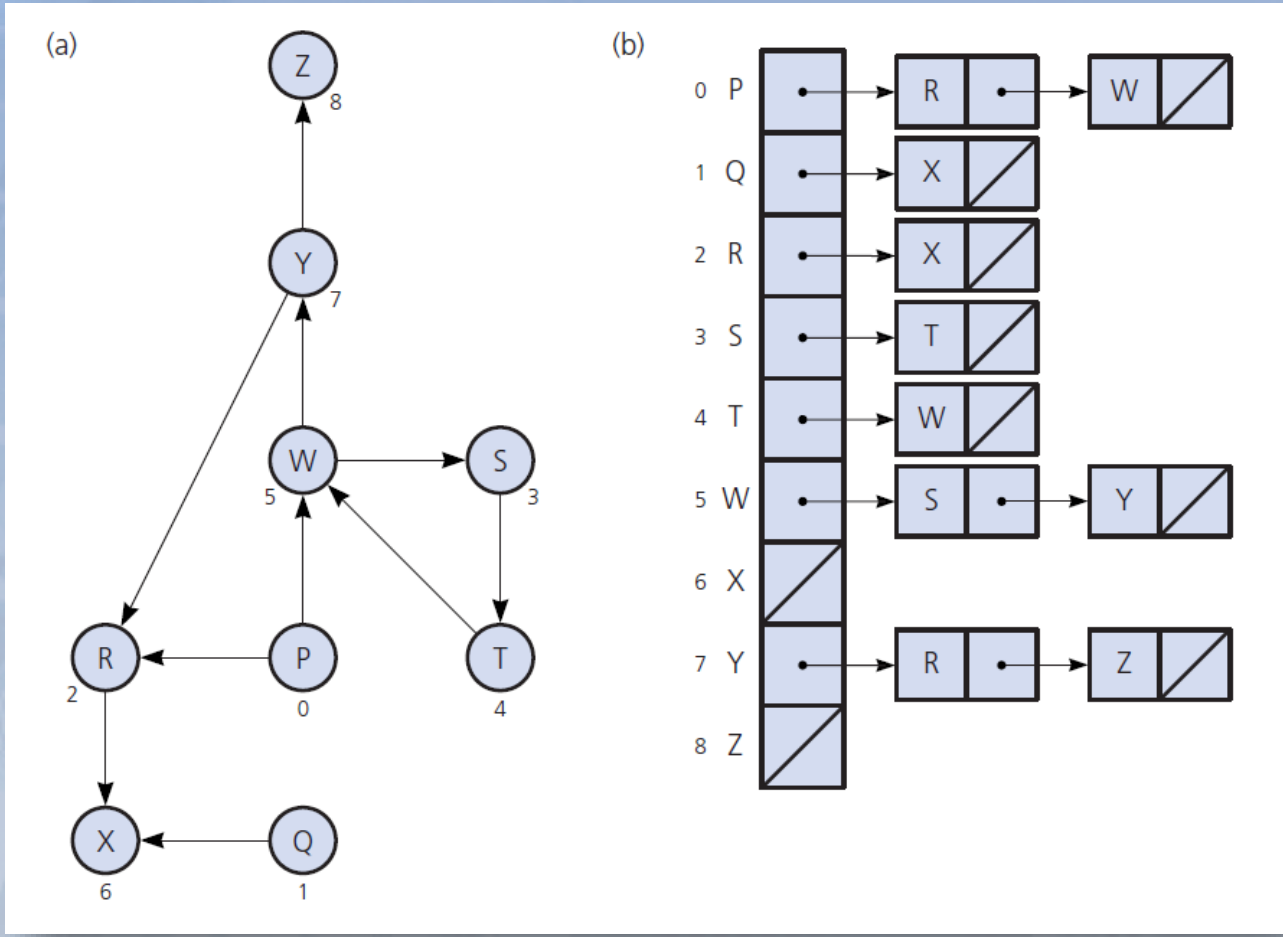


FIGURE 20-9 (a) A directed graph and (b) its adjacency list

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Implementing Graphs

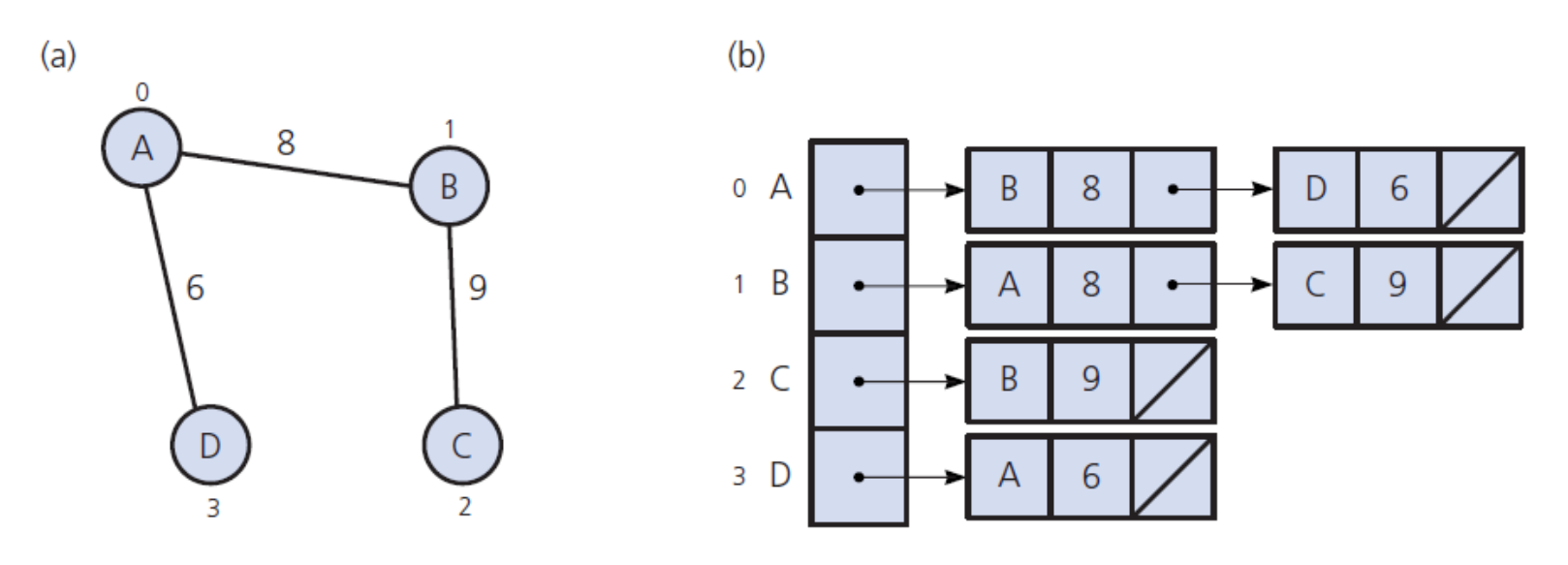


FIGURE 20-10 (a) A weighted undirected graph and (b) its adjacency list

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Implementing Graphs

- Adjacency list
  - Often requires less space than adjacency matrix
  - Supports finding vertices adjacent to given vertex by traversing the ith linked chain to determine whether a vertex corresponding to vertex j is present
- Adjacency matrix
  - Supports process of seeing if there exists an edge from vertex *i* to vertex *j*

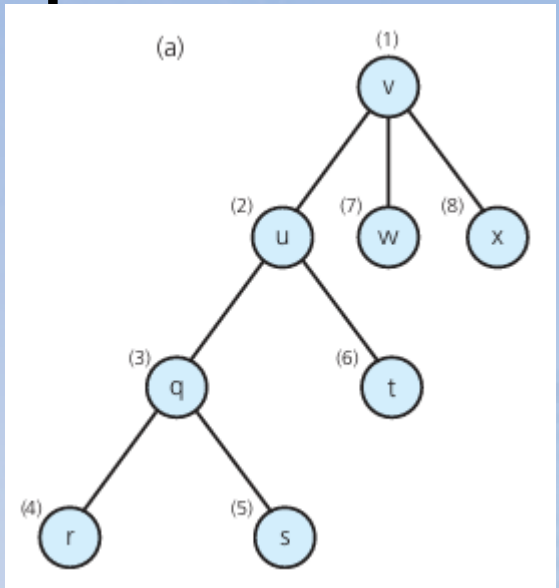
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Graph Traversals

- Visits all vertices it can reach
- Visits all vertices if and only if the graph is connected, regardless where the traversal starts
- Can be used to determine if graph is connected
- Mark/check that a vertex is never visited more than once, to prevent infinite loop, in case of a cycle

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Depth-First Search



- DFS traversal
  - Goes as far as possible from a vertex before backing up

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

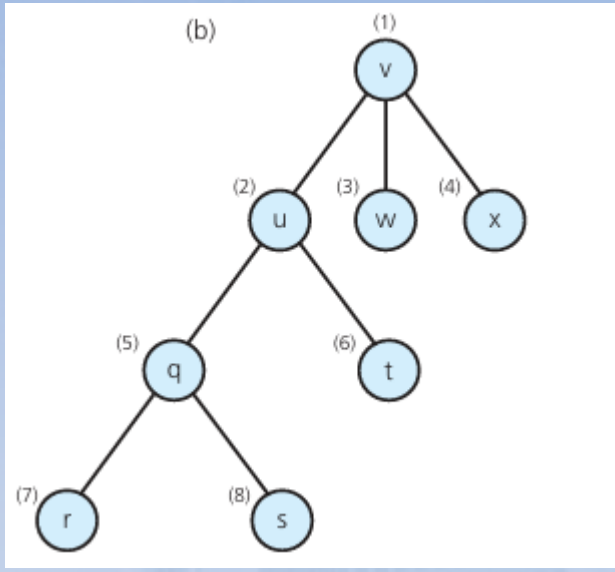
Depth-First Search

- Recursive transversal algorithm

```
// Traverses a graph beginning at vertex v by using a
// depth-first search: Recursive version.
dfs(v: Vertex)
 { Mark v as visited
   for (each unvisited vertex u adjacent to v)
      dfs(u)
 }
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

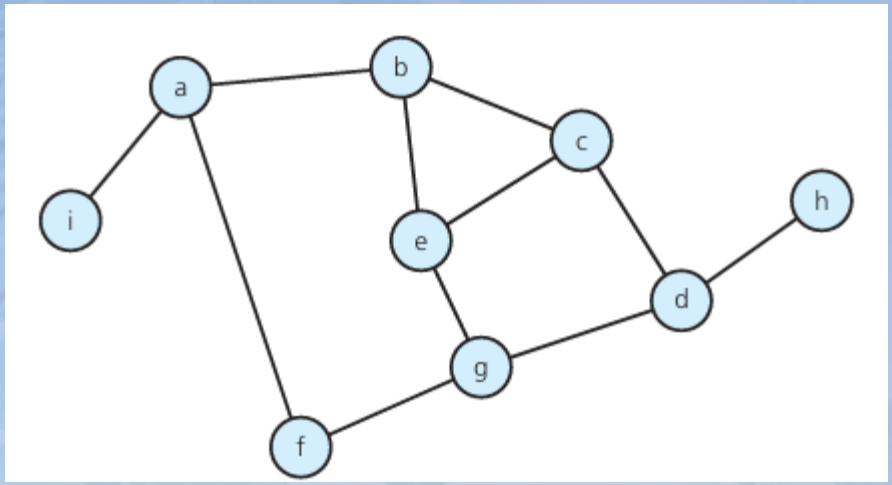
Breadth-First Search



- BFS traversal
  - Visits all vertices adjacent to a vertex before going forward

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Depth-First Search

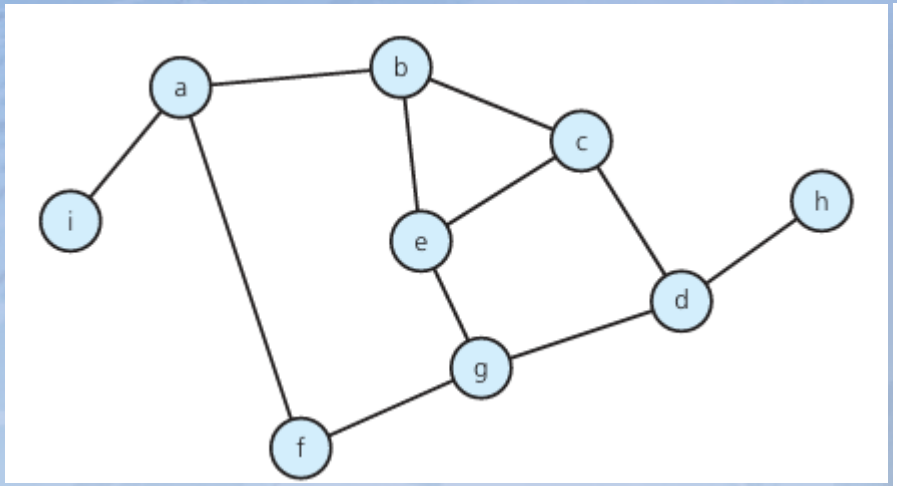


Node visited	Stack (bottom to top)
a	a
b	a b
c	a b c
d	a b c d
g	a b c d g
e	a b c d g e
(backtrack)	a b c d g
f	a b c d g f
(backtrack)	a b c d g
(backtrack)	a b c d
h	a b c d h
(backtrack)	a b c d
(backtrack)	a b c
(backtrack)	a b
(backtrack)	a
i	a i
(backtrack)	a
(backtrack)	*(empty)*

The results of a depth-first traversal, beginning at vertex a.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Breadth-First Search



Node visited	Queue (front to back)
a	a
	(empty)
b	b
f	b f
i	b f i
	f i
c	f i c
e	f i c e
	i c e
g	i c e g
	c e g
	e g
d	e g d
	g d
	d
	(empty)
h	h
	(empty)

The results of a breadth-first traversal, beginning at vertex a.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

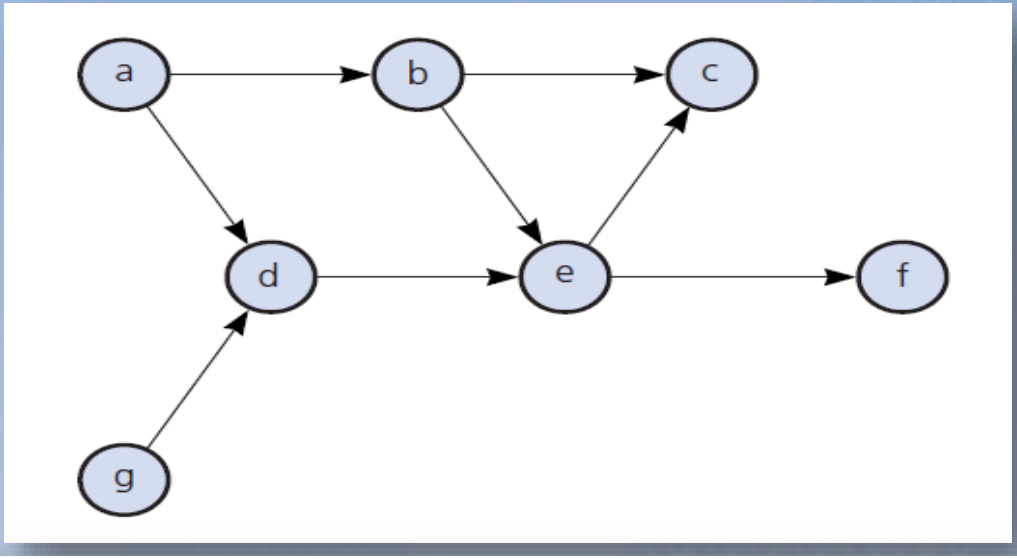
Applications of Graphs

- Topological Sorting
- Spanning Trees
- Minimum Spanning Trees
- Shortest Paths
- Circuits
- Some Difficult Problems

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

- A topological order is a linear order of the vertices in a directed graph without cycles.
- All of the edges point in one direction, if you arrange the vertices of digraph linearly and in topological order.
- Arranging the vertices into a topological order is called topological sorting.



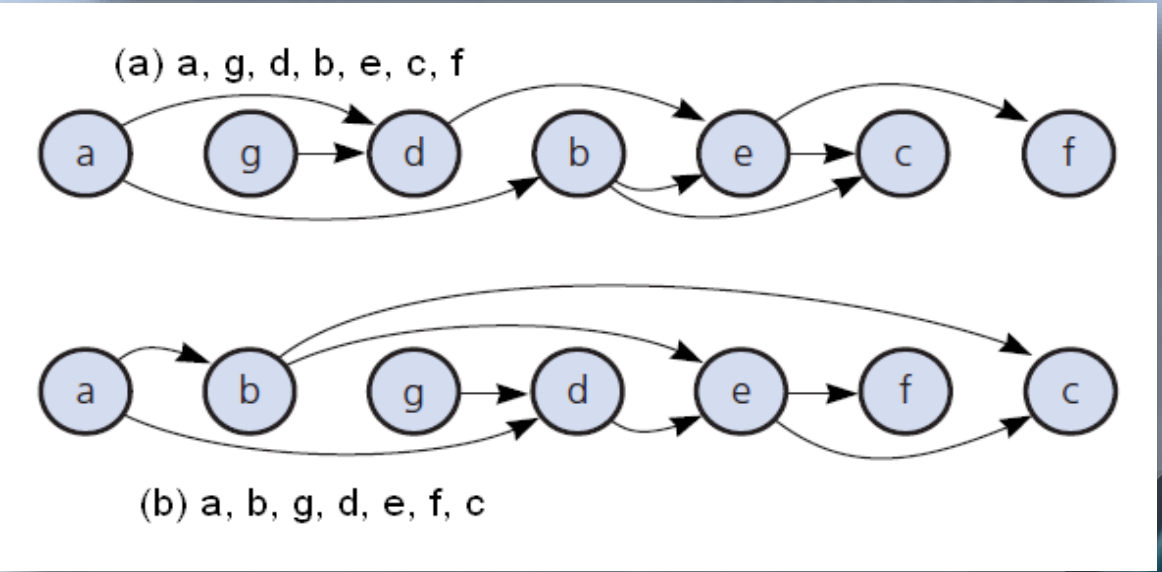


FIGURE 20-15 A directed graph without cycles

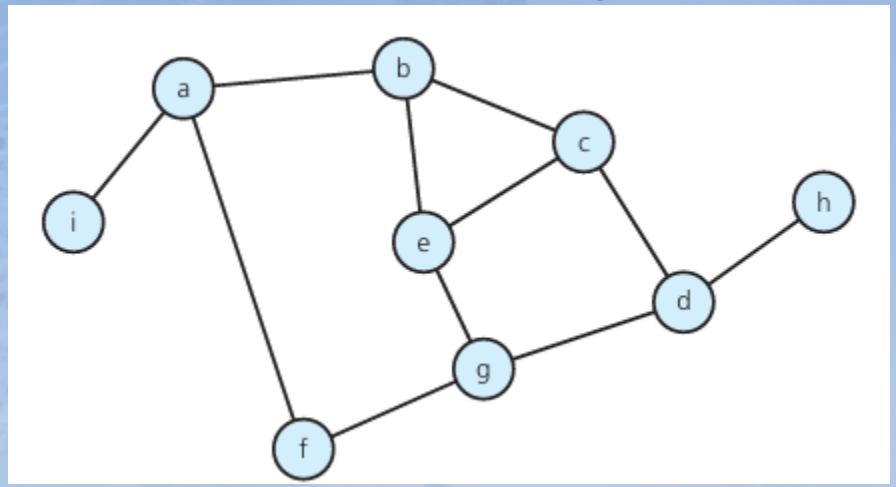
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Spanning Trees

- A tree is an undirected connected graph without cycles
- The nodes (vertices) of a tree have a hierarchical arrangement that is not required of all graphs
- Detecting a cycle in an undirected graph
  - Connected undirected graph with *n* vertices must have at least *n* – 1 edges
  - If it has exactly *n* – 1 edges, it cannot contain a cycle
  - With more than *n* – 1 edges, must contain at least one cycle

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Spanning Trees



- There may be several spanning trees for a given graph.
- If you have a connected undirected graph with cycles and you remove edges until there are no cycles, you will obtain a spanning tree for the graph.

The dashed lines in Figure 20-19 indicate edges that were omitted from the graph to form the tree.

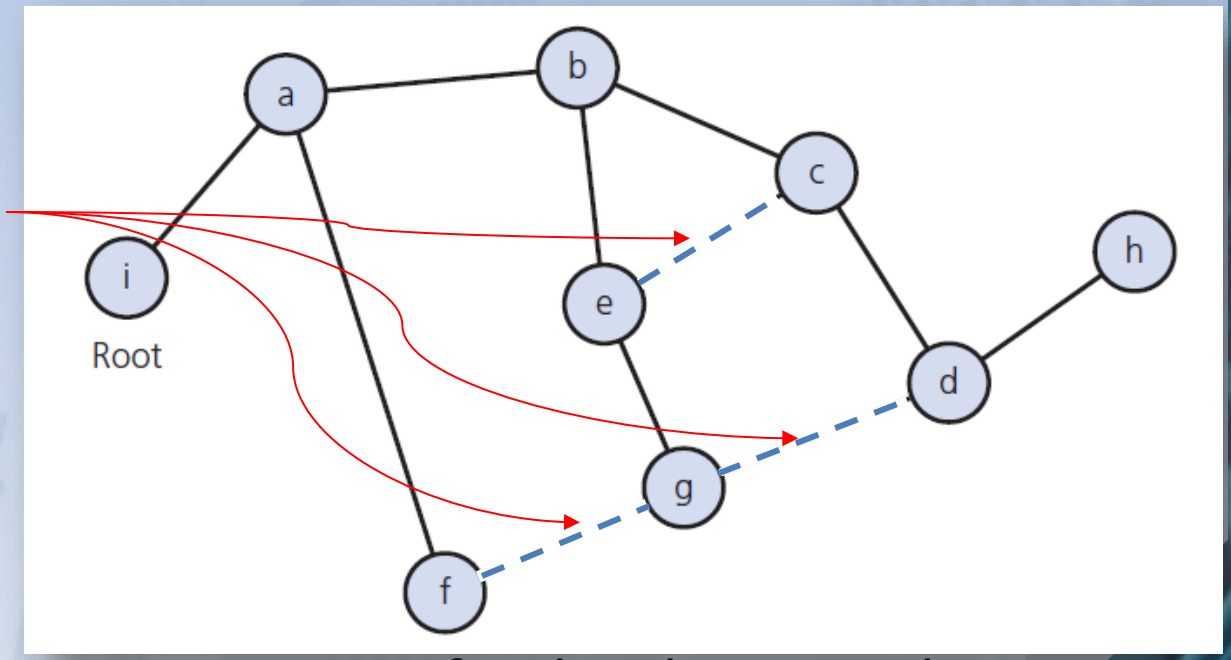


FIGURE 20-19 A spanning tree for the above graph

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Spanning Trees

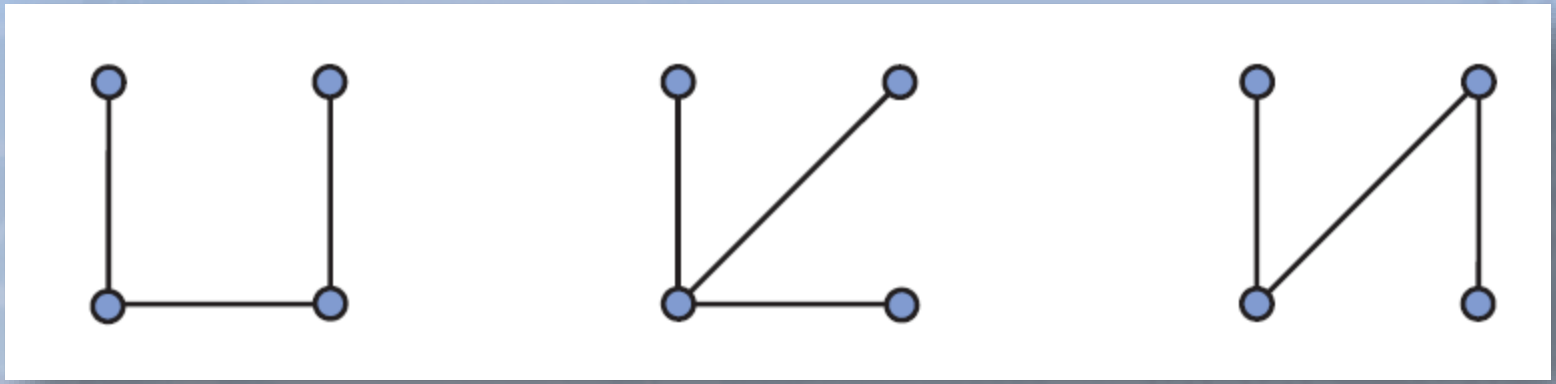


FIGURE 20-20 Connected graphs that each have four vertices and three edges

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

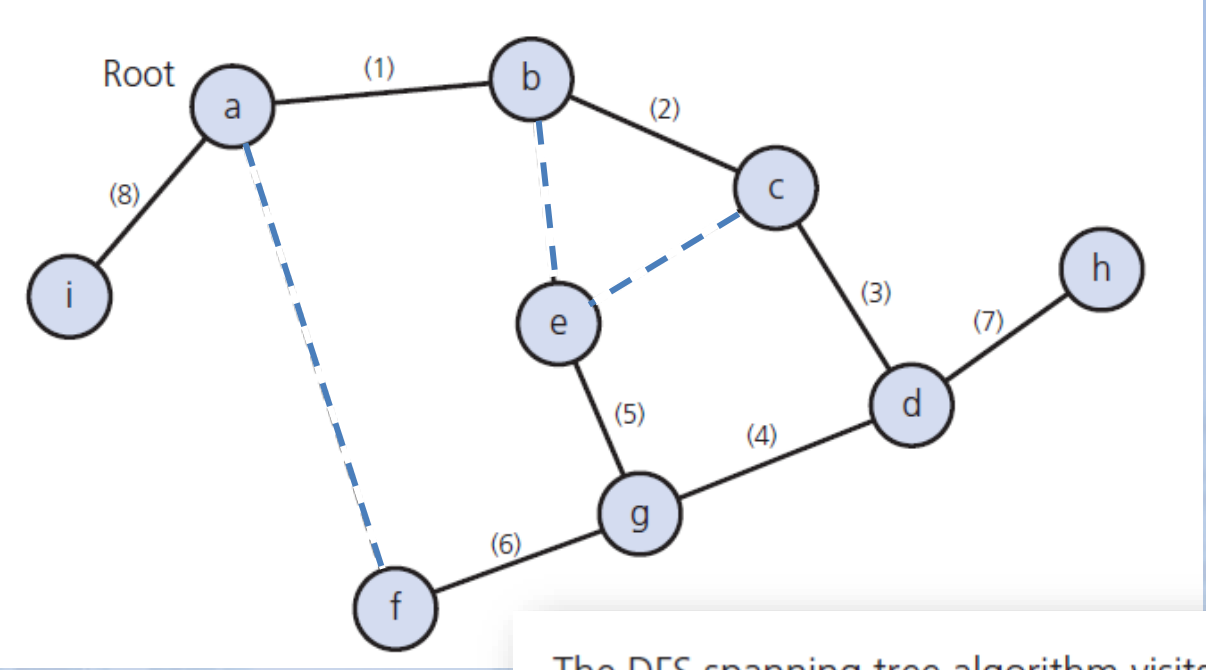
Spanning Trees

```
// Forms a spanning tree for a connected undirected graph
// beginning at vertex v by using depth-first search:
// Recursive version.
dfsTree(v: Vertex)

   Mark v as visited

   for (each unvisited vertex u adjacent to v)
   {
      Mark the edge from u to v
      dfsTree(u)
   }
```

DFS spanning tree algorithm

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Spanning Trees



The DFS spanning tree algorithm visits vertices in this order: a, b, c, d, g, e, f, h, i. Numbers indicate the order in which the algorithm marks edges.

FIGURE 20-21 The DFS spanning tree rooted at vertex a

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Spanning Trees

```
// Forms a spanning tree for a connected undirected graph
// beginning at vertex v by using breadth-first search:
// Iterative version.
bfsTree(v: Vertex)

   q = a new empty queue

   // Add v to queue and mark it
   q.enqueue(v)
   Mark v as visited

   while (!q.isEmpty())
   {
      q.dequeue(w)

      // Loop invariant: there is a path from vertex w to
      // every vertex in the queue q
      for (each unvisited vertex u adjacent to w)
      {
         Mark u as visited
         Mark edge between w and u
         q.enqueue(u)
      }
   }
```

BFS spanning tree algorithm

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Spanning Trees

The BFS spanning tree algorithm visits vertices in this order: a, b, f, i, c, e, g, d, h. Numbers indicate the order in which the algorithm marks edges.

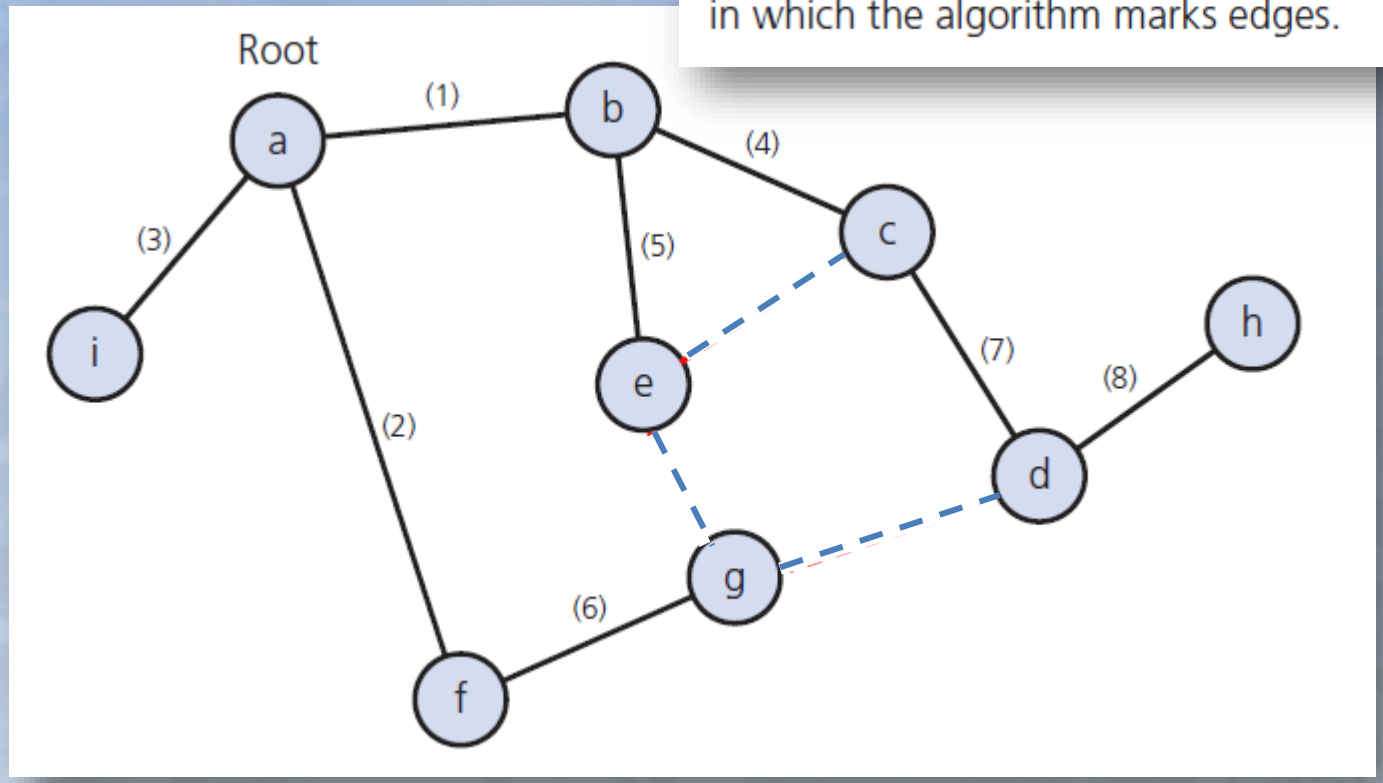
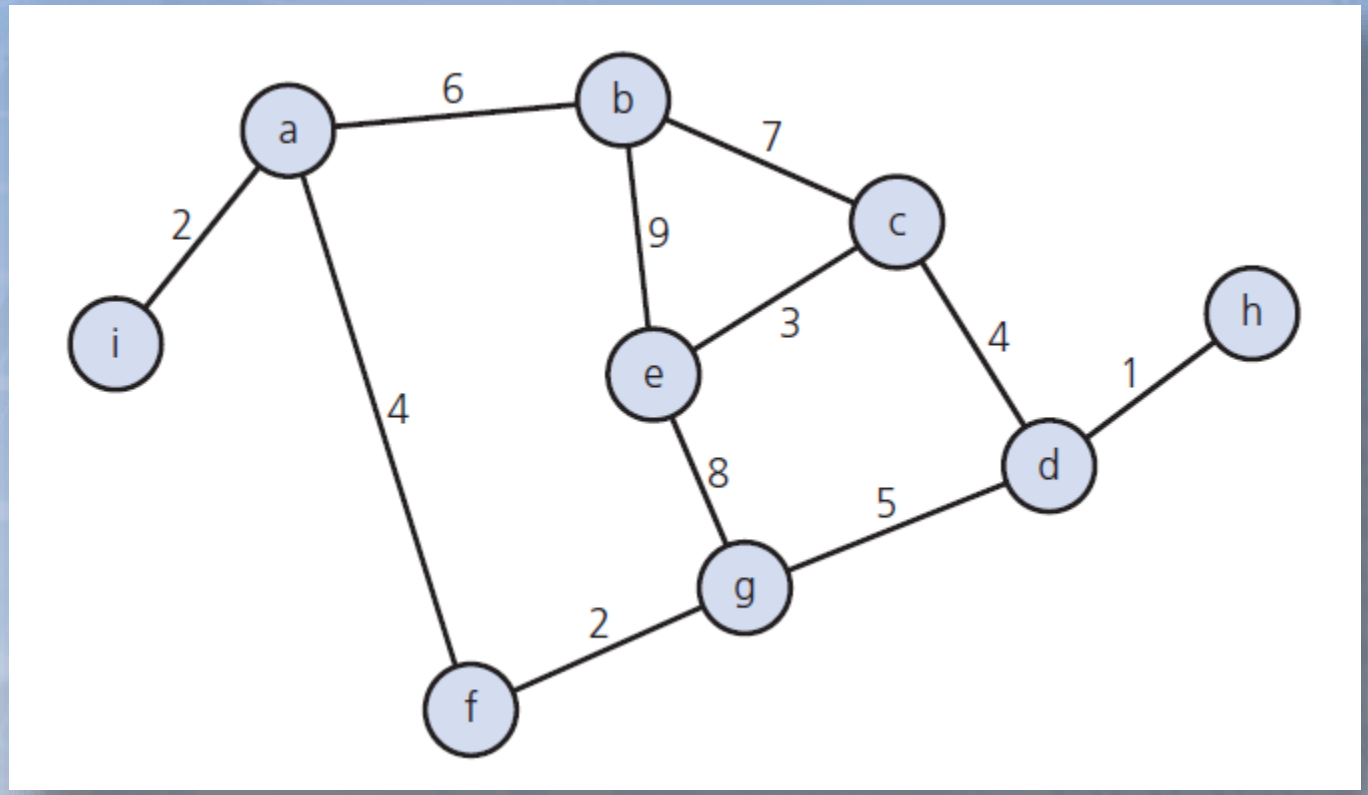


FIGURE 20-23 The BFS spanning tree rooted at vertex *a*

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Minimum Spanning Trees



- A minimum spanning tree of a connected undirected graph has a minimal edge-weight sum.
- A minimum spanning tree need not be unique, since there may be several minimum spanning trees for a particular graph, where their costs are equal.

FIGURE 20-24 A weighted, connected, undirected graph

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Minimum Spanning Trees

```
// Determines a minimum spanning tree for a weighted,
// connected, undirected graph whose weights are
// nonnegative, beginning with any vertex v.
primsAlgorithm(v: Vertex)

   Mark vertex v as visited and include it in the minimum spanning tree
   while (there are unvisited vertices)
   {
       Find the least-cost edge (v, u) from a visited vertex v to some unvisited vertex u
       Mark u as visited
       Add the vertex u and the edge (v, u) to the minimum spanning tree
   }
```

Prim's algorithm is a **greedy algorithm** that finds a minimum spanning tree for a weighted undirected graph. (https://en.wikipedia.org/wiki/Prim%27s_algorithm)

1. Initialize a tree with a single vertex, chosen arbitrarily from the graph.
2. Grow the tree by one edge: Of the edges that connect the tree to vertices not yet in the tree, find the minimum-weight edge, and transfer it to the tree.
3. Repeat step 2 (until all vertices are in the tree).

Prims algorithm

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Minimum Spanning Trees

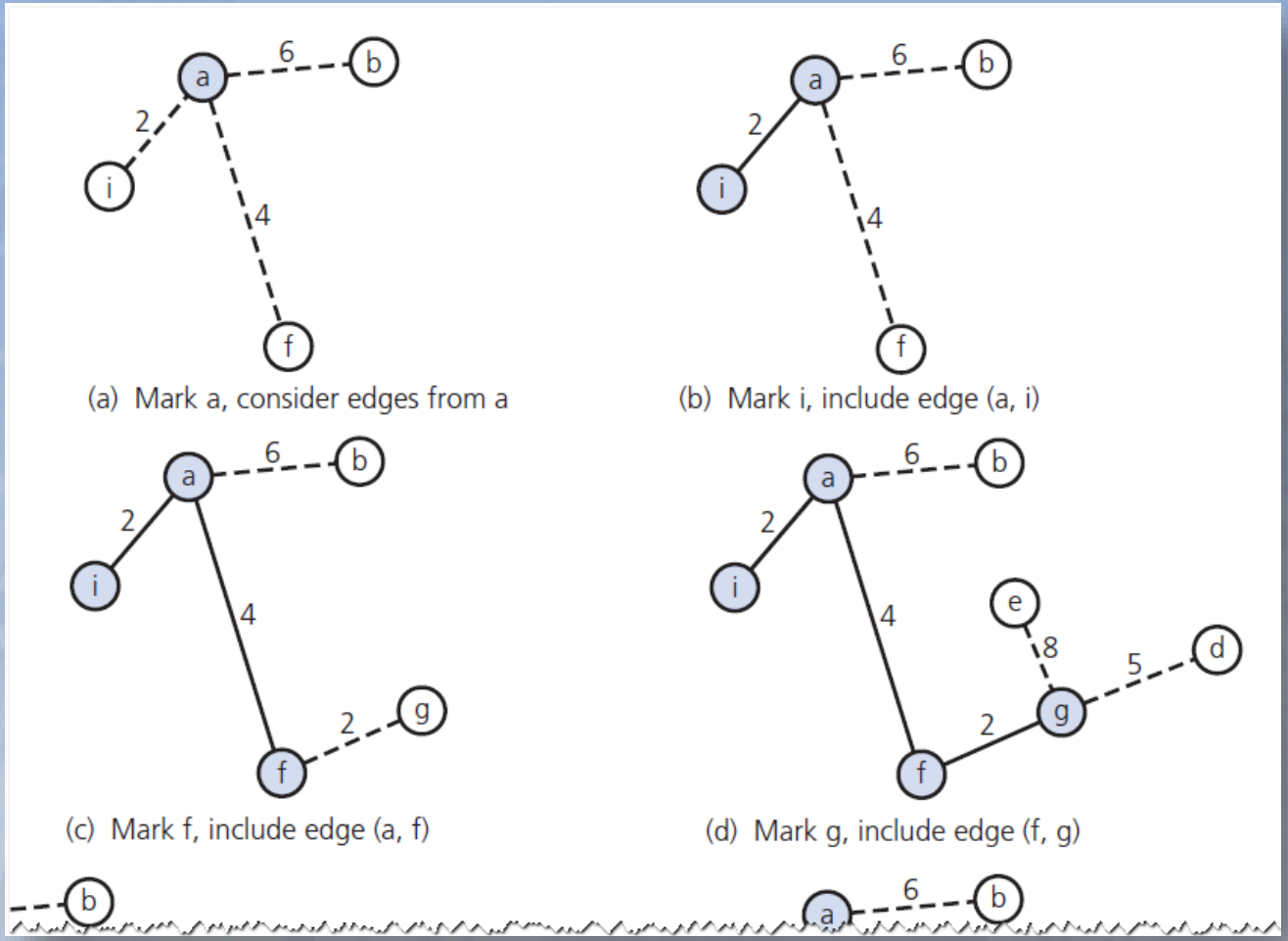


FIGURE 20-25 A trace of primsAlgorithm for the graph in Figure 20-23, beginning at vertex a

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Minimum Spanning Trees

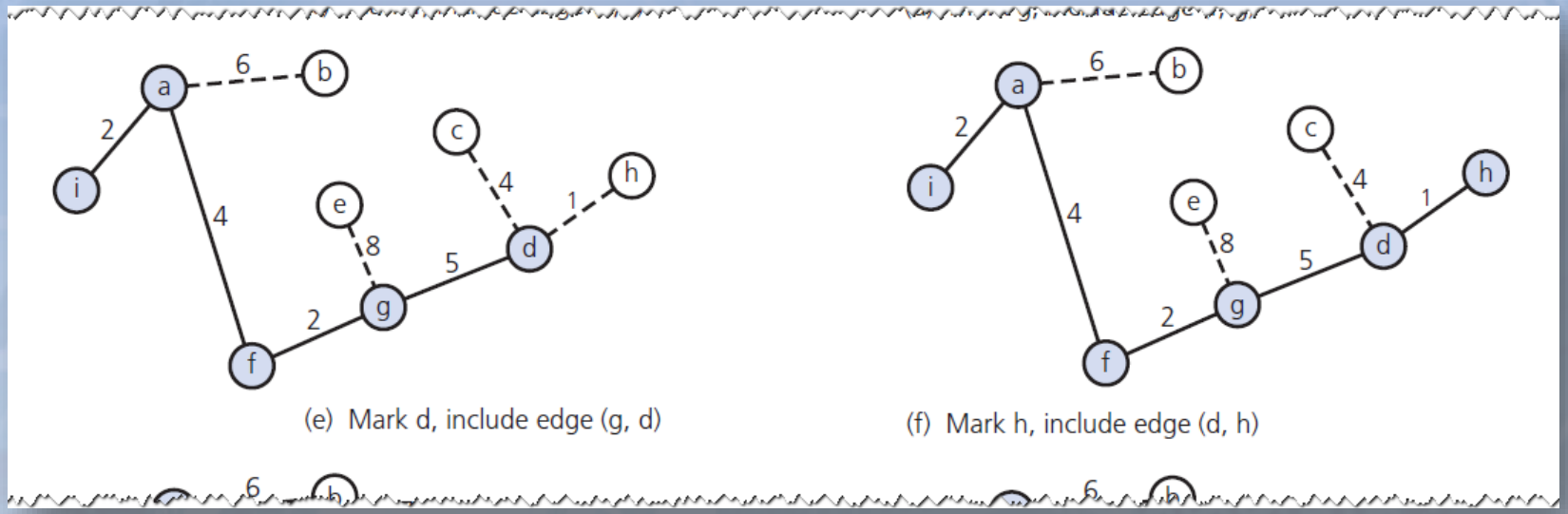


FIGURE 20-25 A trace of primsAlgorithm for the graph in Figure 20-23, beginning at vertex a

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Minimum Spanning Trees

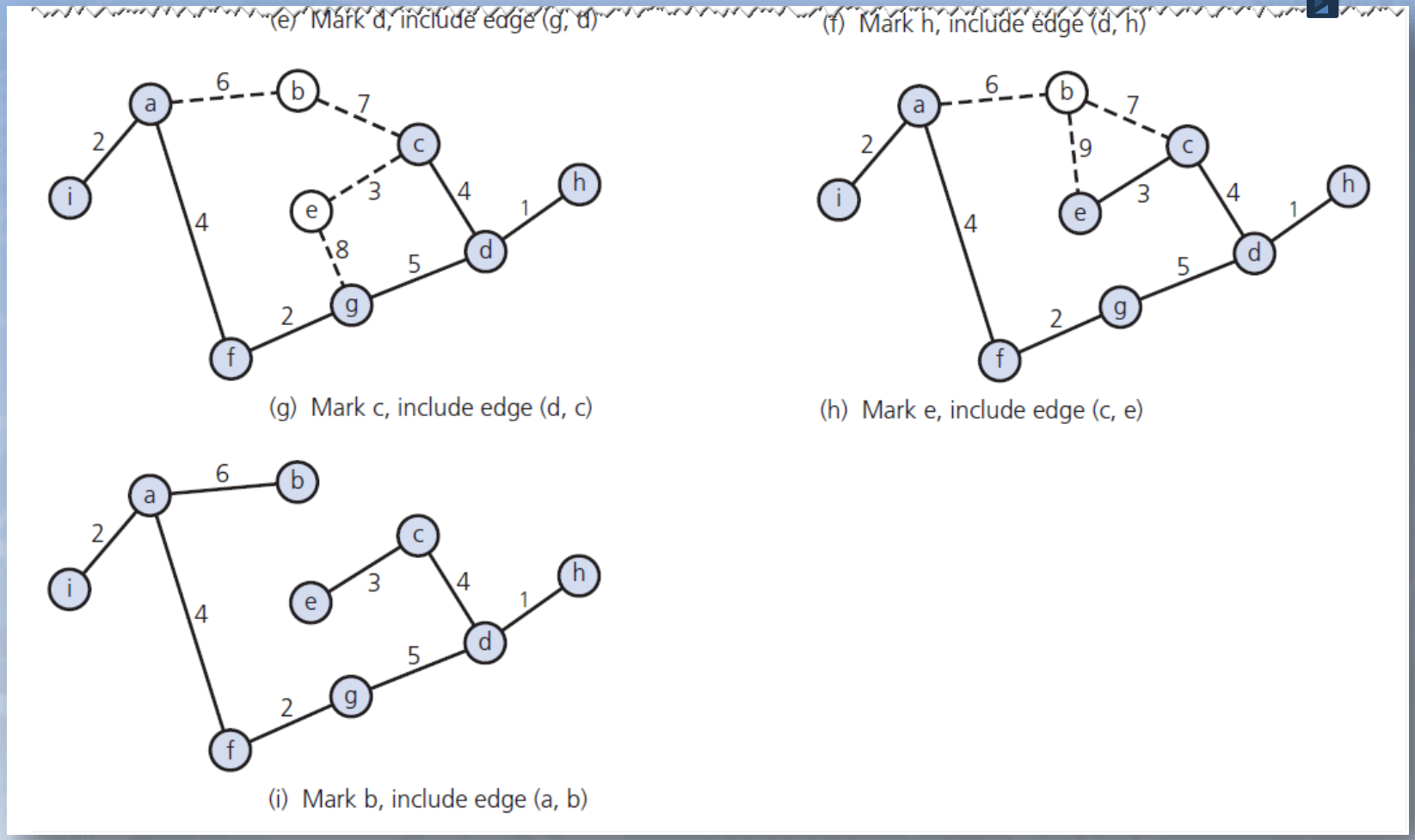
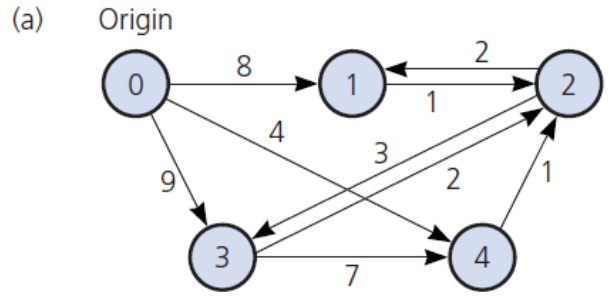


FIGURE 20-25 A trace of primsAlgorithm for the graph in Figure 20-23, beginning at vertex a

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Shortest Paths

- The shortest path between two vertices in a weighted graph
  - Has the smallest edge-weight sum



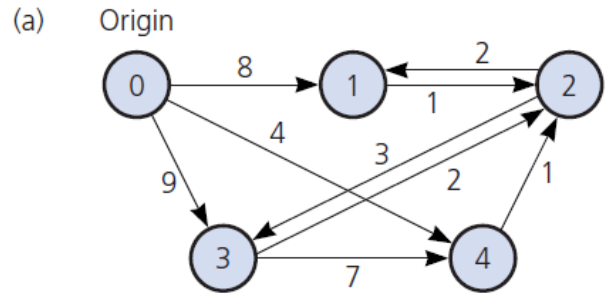
(b)

	0	1	2	3	4
0	∞	8	∞	9	4
1	∞	∞	1	∞	∞
2	∞	2	∞	3	∞
3	∞	∞	2	∞	7
4	∞	∞	1	∞	∞

FIGURE 20-26 (a) A weighted directed graph and (b) its adjacency matrix

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Shortest Paths



(b)

	0	1	2	3	4
0	∞	8	∞	9	4
1	∞	∞	1	∞	∞
2	∞	2	∞	3	∞
3	∞	∞	2	∞	7
4	∞	∞	1	∞	∞

Dijkstra's algorithm(1956, 1959) finds the shortest path from a given source node to every other node.

			weight				
Step	v	vertexSet	[0]	[1]	[2]	[3]	[4]
1	–	0	0	8	∞	9	4
2	4	0, 4	0	8	5	9	4
3	2	0, 4, 2	0	7	5	8	4
4	1	0, 4, 2, 1	0	7	5	8	4
5	3	0, 4, 2, 1, 3	0	7	5	8	4

FIGURE 20-27 A trace of the shortest-path (Dijkstra) algorithm applied to the graph in Figure 20-26a

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Shortest Paths

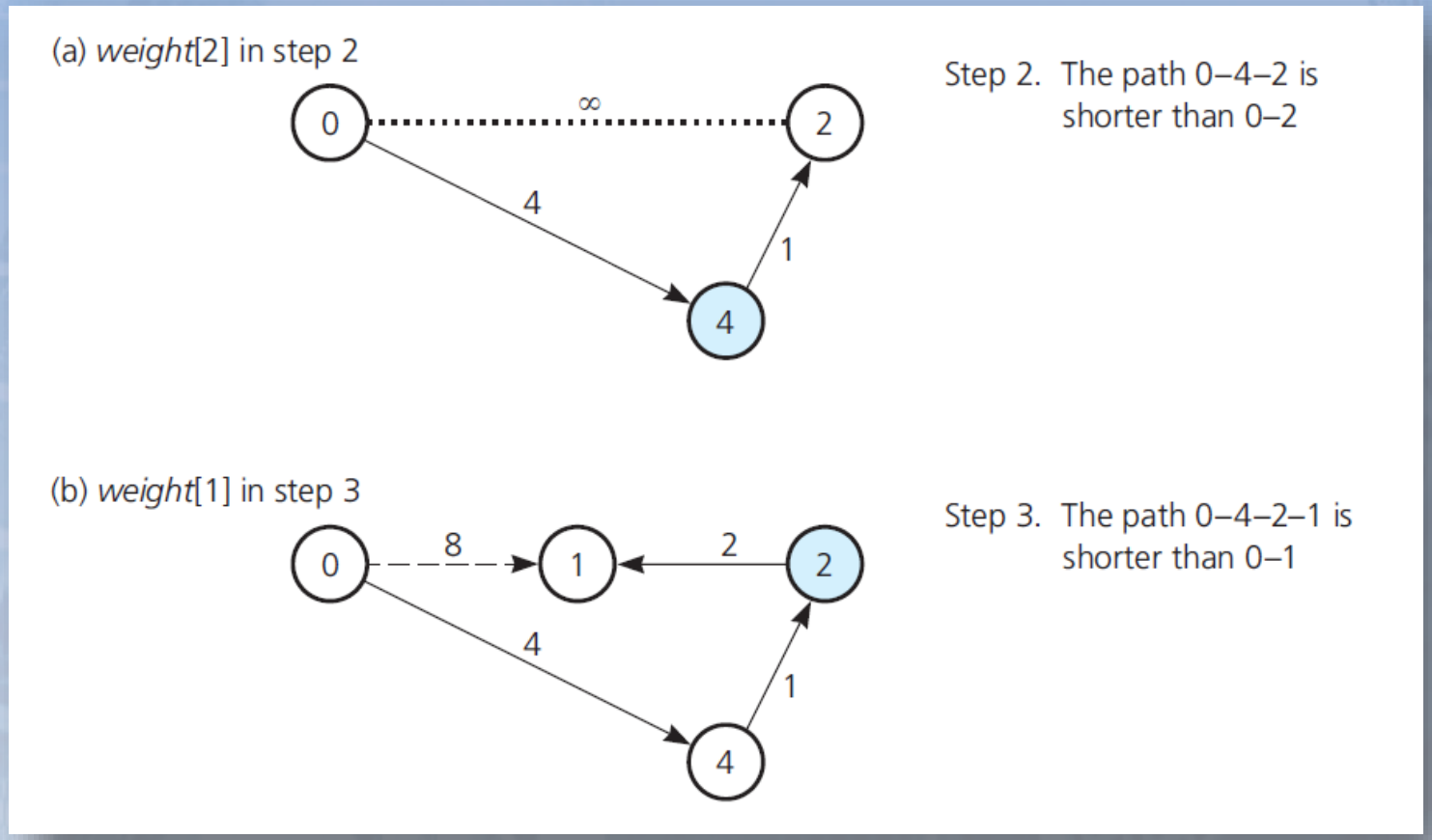


FIGURE 20-28 Checking weight[u] by examining the graph: (a) weight[2] in step 2; (b) weight[1] in step 3;

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Shortest Paths

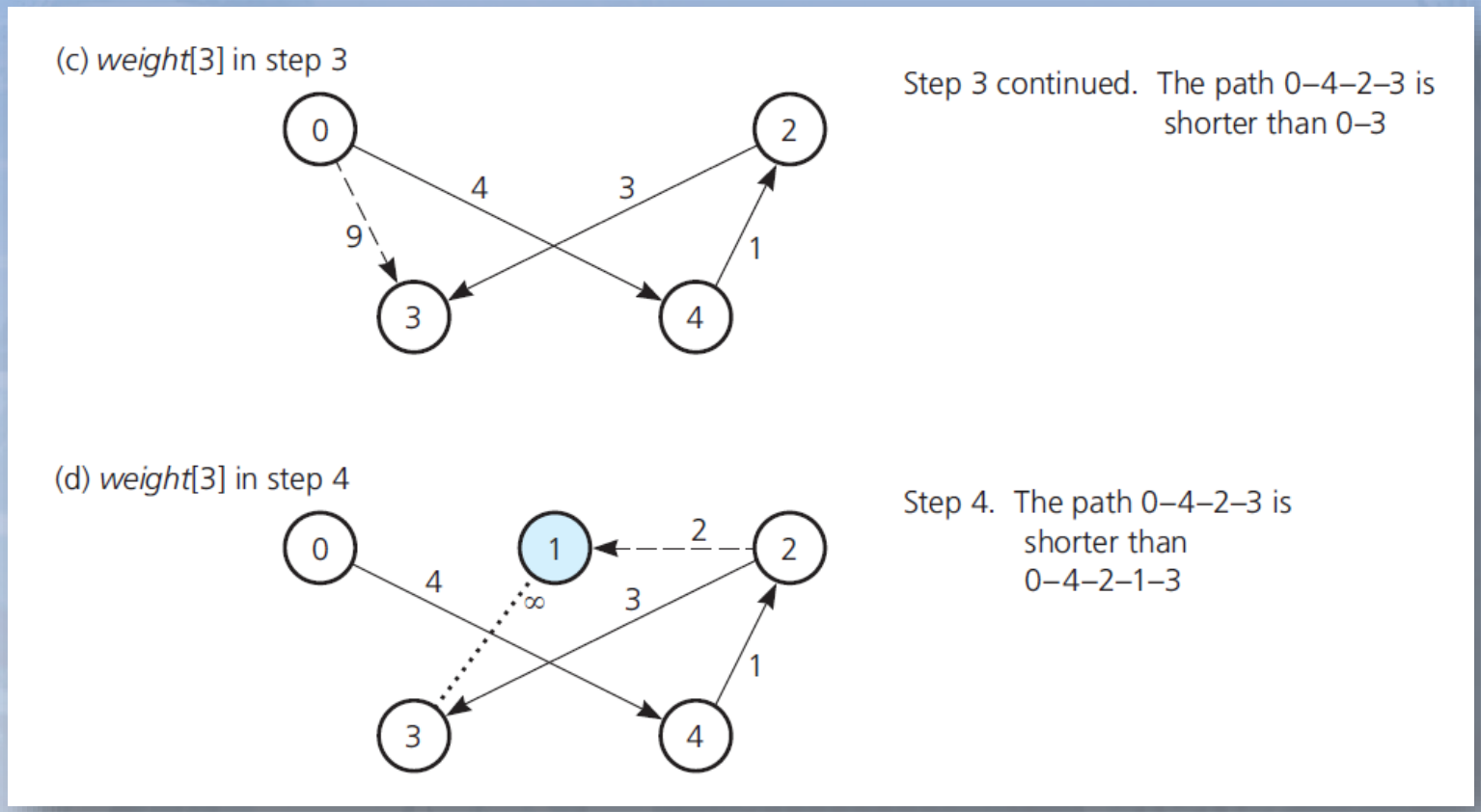


FIGURE 20-28 Checking weight[u] by examining the graph: (c) weight[3] in step 3; (d) weight[3] in step 4

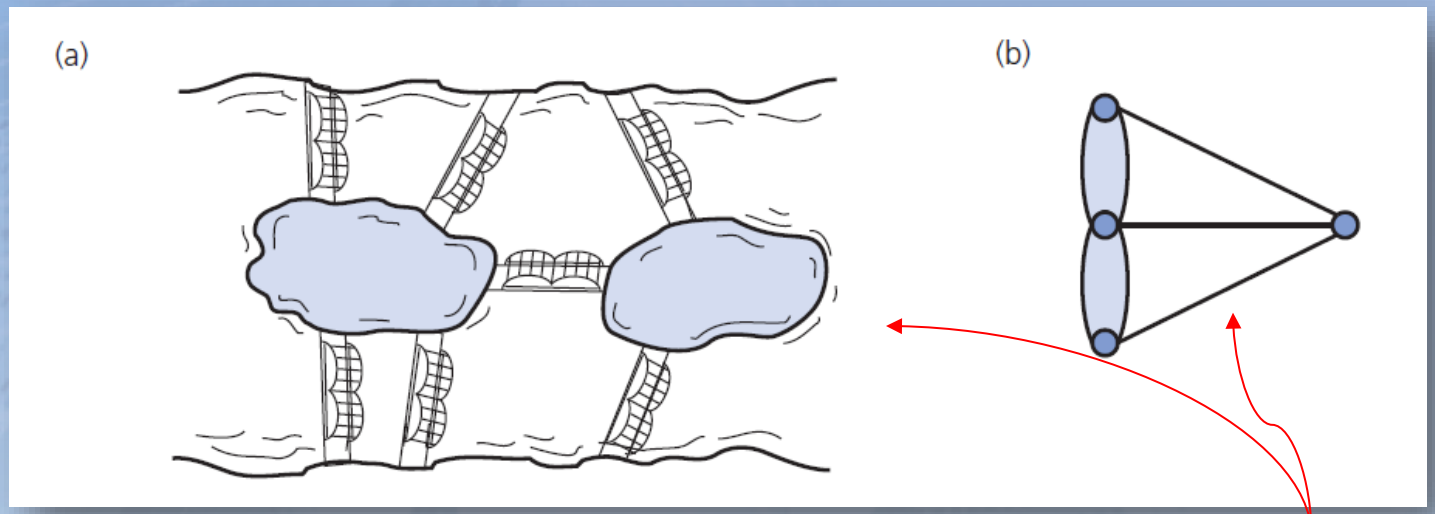
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits

- Circuit
  - Another name for type of cycle common in statement of certain types of problems
  - Typical circuits either visit every vertex once or every edge once
- Euler Circuit
  - Begins at vertex *v*
  - Passes through every edge exactly once
  - Terminates at *v*

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits



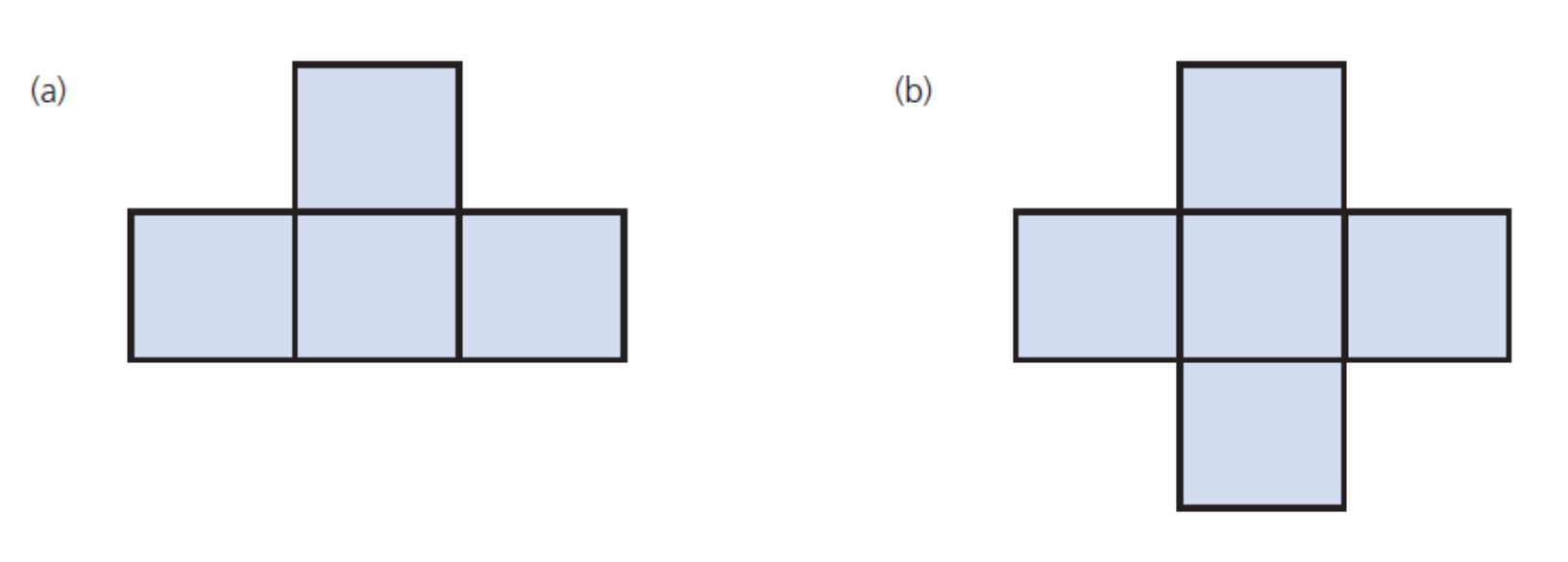
Euler demonstrated that no solution exists for this particular configuration of edges and vertices.

- A path in an undirected graph that begins at a vertex v, passes through every edge in the graph exactly once, and terminates at v is called an Euler circuit.
- An Euler circuit exists if and only if each vertex touches an even number of edges.

FIGURE 20-28 (a) Euler's bridge problem and (b) its multigraph representation

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits



Finding an Euler circuit is like drawing each of the diagrams in Figure 20-29 (a) without lifting your pencil or redrawing a line, and ending at your starting point.

No solution is possible for Figure 20-29 (a).

You can find one for Figure 20-29 (b).

FIGURE 20-29 Pencil and paper drawings

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Some Difficult Problems

- Hamilton circuit
  - Begins at vertex *v*
  - Passes through every vertex exactly once
  - Terminates at *v*
- Variation is "traveling salesperson problem"
  - Visit every city on his route exactly once
  - Edge (road) has associated cost (mileage)
  - Goal is determine least expensive circuit

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traveling Salesperson Problem

- Famous problem which has not been solved efficiently.
- There are numerous algorithms
  - Brute Force Algorithm O(n!)
    - generate all possible city permutations and calculate each route's length
  - Nearest Neighbor Algorithm $O(n^2)$
    - starts at a random city and repeatedly visits the nearest city until all have been visited
    - Approximation
  - Bellman-Held-Karp algorithm $O(2^n * n^2)$
    - Fastest known algorithm for solving general problem

https://www.networkpages.nl/a-big-breakthrough-in-the-euclidean-travelling-salesman-problem/#:~:text=For%20the%20Travelling%20Salesman%20Problem,to%20come%20to%20an%20answer.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traveling Salesperson Problem

- Bellman-Held-Karp algorithm $O(2^n * n^2)$
- Assume n = 100 cities
- Perform roughly $2^n * n^2$ steps to come to an answer.
- It takes computer roughly $2^{100} *10{,}000=2 * 10^{33}$ (a number with 33 digits) steps to give an answer using this algorithm.
- Consider you have a 2GHz processor, then this performs $2 * 10^9$ operations per second.
- This processor needs 10^{24} seconds to give an answer to this problem.
- How many years are there in 10^{24} seconds?
  - ~3.17×10^{16} years
  - The age of the universe is estimated to be 13.8 billion years, or ~ 1.38×10^{10} years.
- Companies use algorithms that can solve specific instances of TSP very fast.
- They don't solve the general problem but they use the structure of a specific case to solve it fast.
- In practice these algorithms give solutions that are optimal or almost optimal, but that is not guaranteed to be the case.
- Fastest known algorithm for solving general problem

https://www.networkpages.nl/a-big-breakthrough-in-the-euclidean-travelling-salesman-problem/#:~:text=For%20the%20Travelling%20Salesman%20Problem,to%20come%20to%20an%20answer.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Chapter 20

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Chapter 20

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Terminology

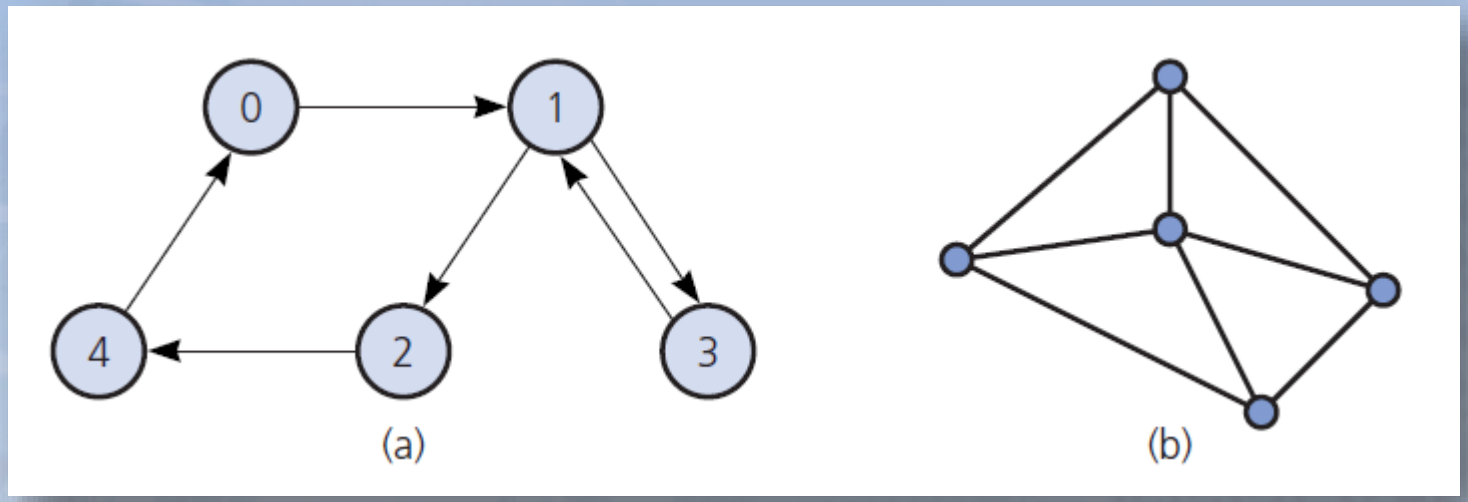


FIGURE 20-6 Graphs for Checkpoint Questions 1, 3, 4, 5

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Depth-First Search

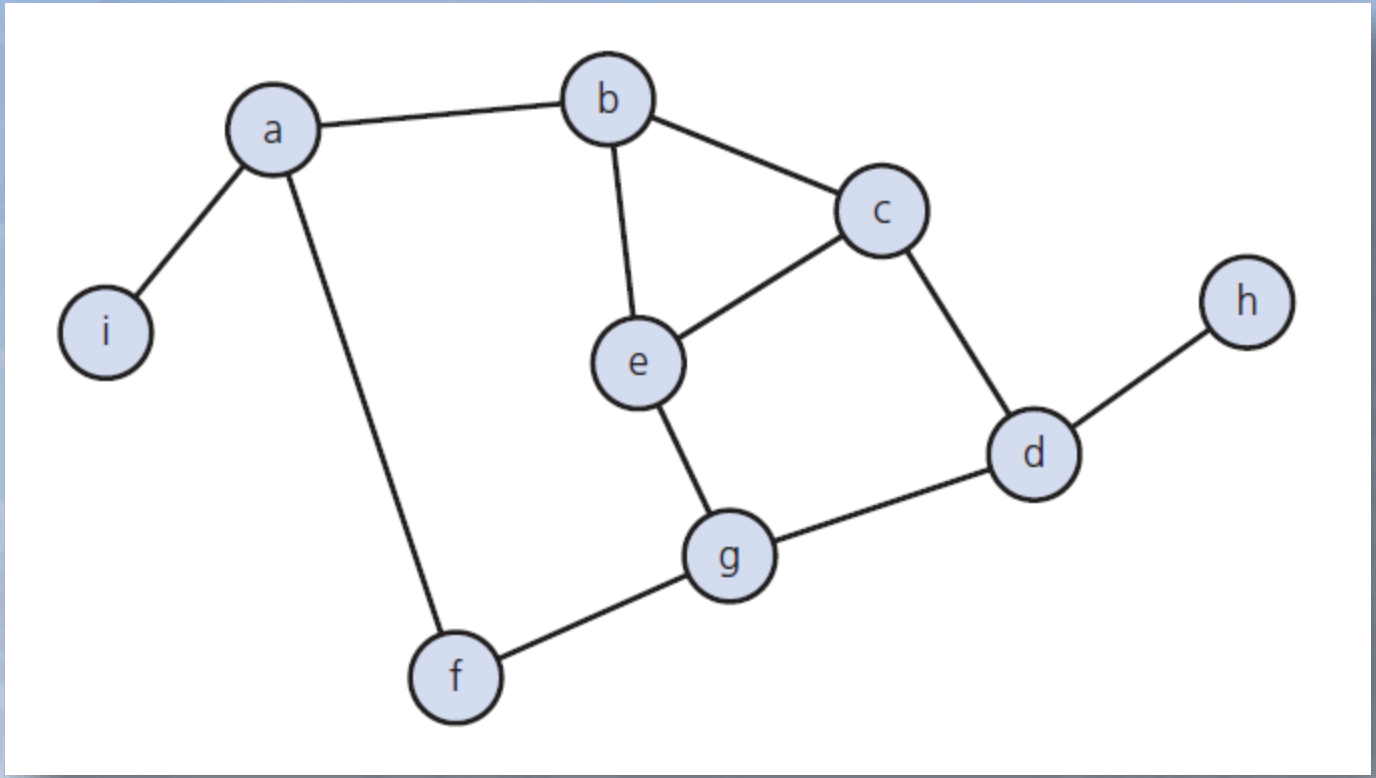


FIGURE 20-12 A connected graph with cycles

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

```
// Arranges the vertices in graph theGraph into a
// topological order and places them in list aList.
topSort1(theGraph: Graph, aList: List)

   n = number of vertices in theGraph
   for (step = 1 through n)
   {
      Select a vertex v that has no successors
      aList.insert(1, v)
      Remove from theGraph vertex v and its edges
   }
```

FIGURE 20-16 Topological Sort Pseudocode

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

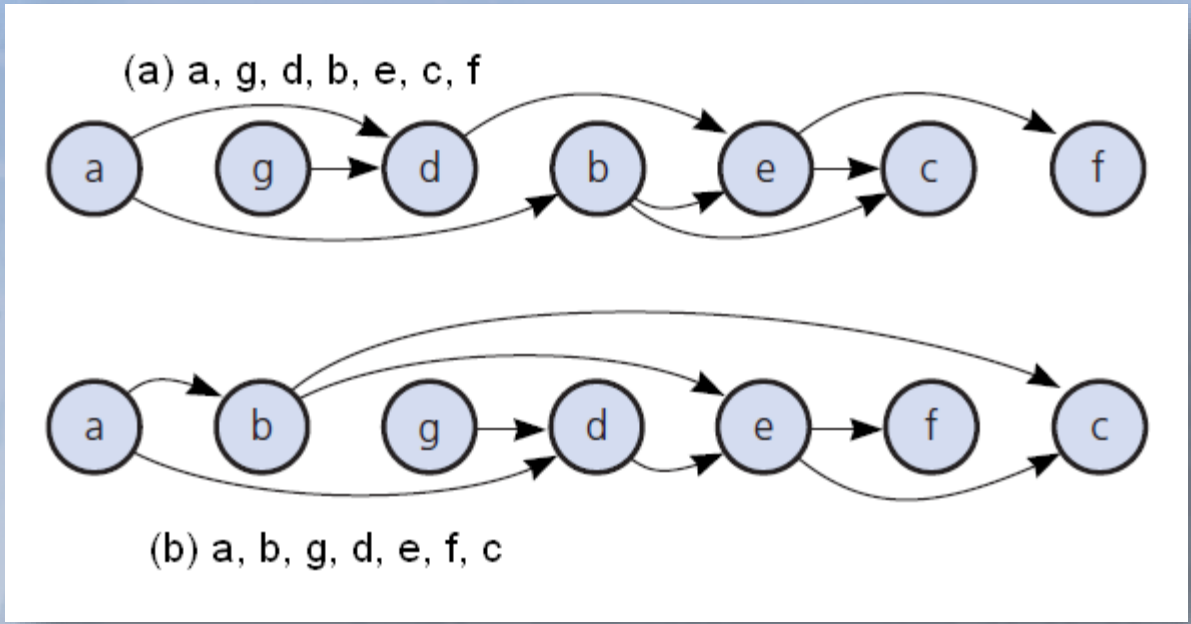


FIGURE 20-16 The graph in Figure 20-15 arranged according to two topological orders

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

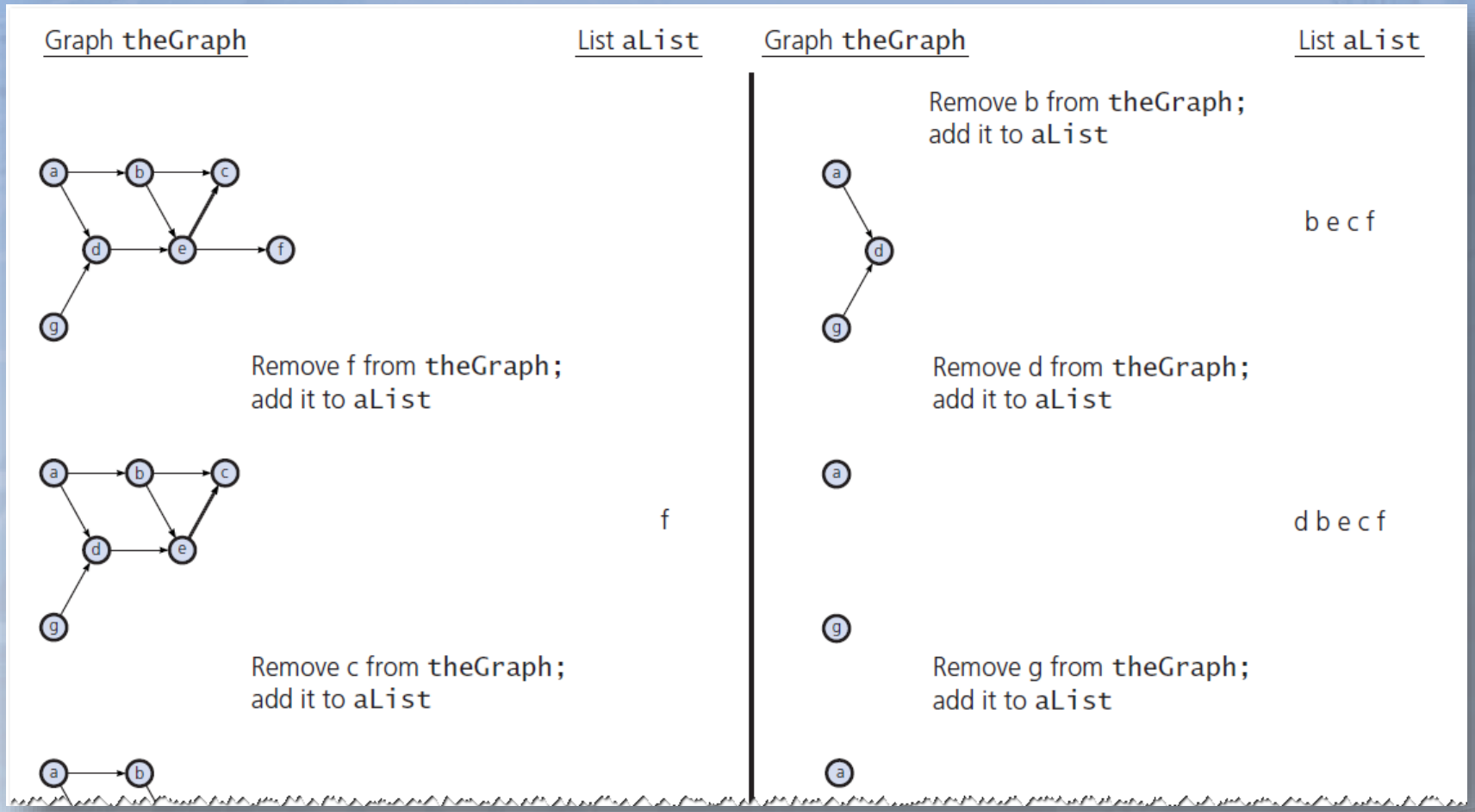


FIGURE 20-17 A trace of topSort1 for the graph in Figure 20-15

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

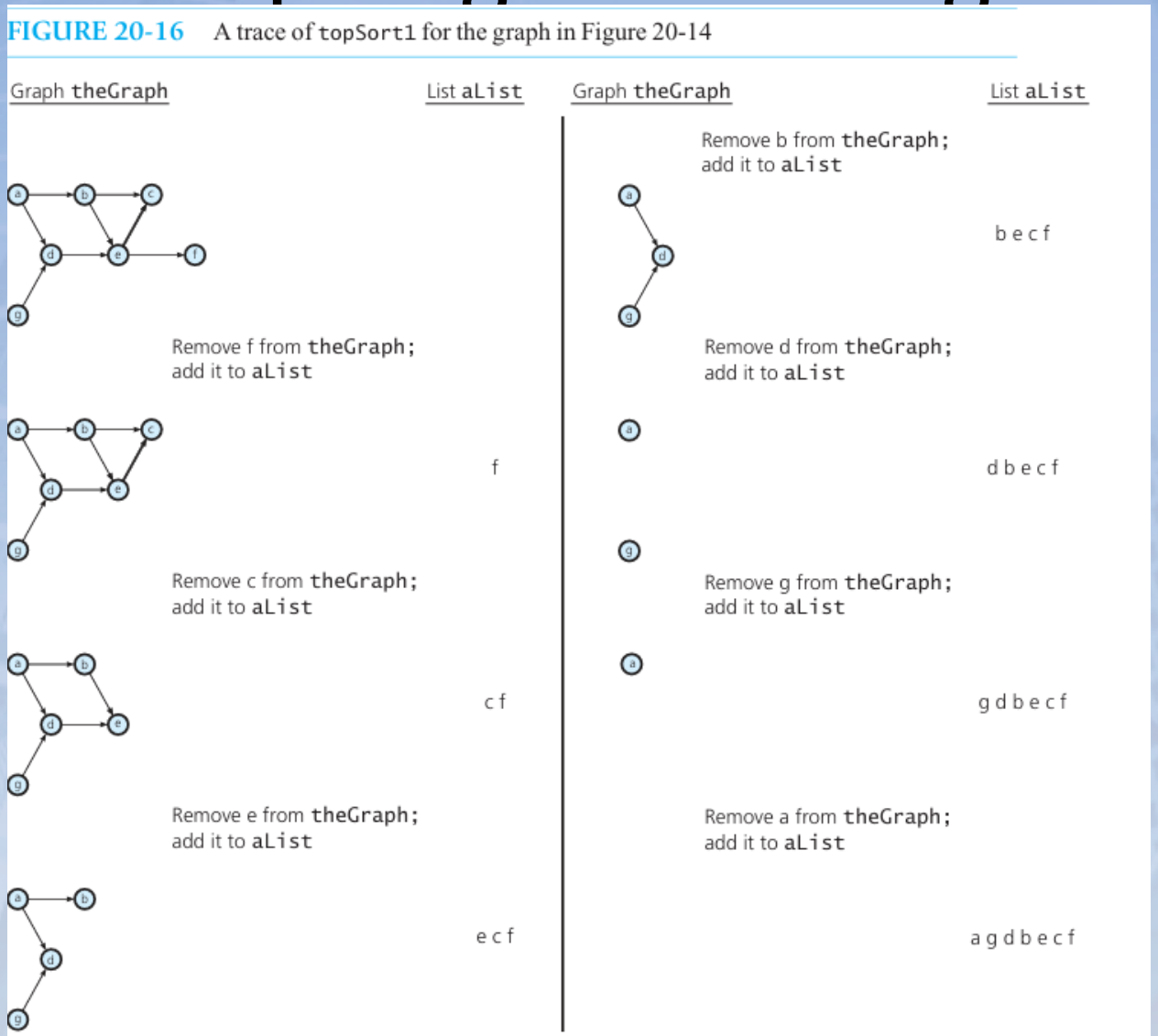


FIGURE 20-17 A trace of topSort1 for the graph in Figure 20-15

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

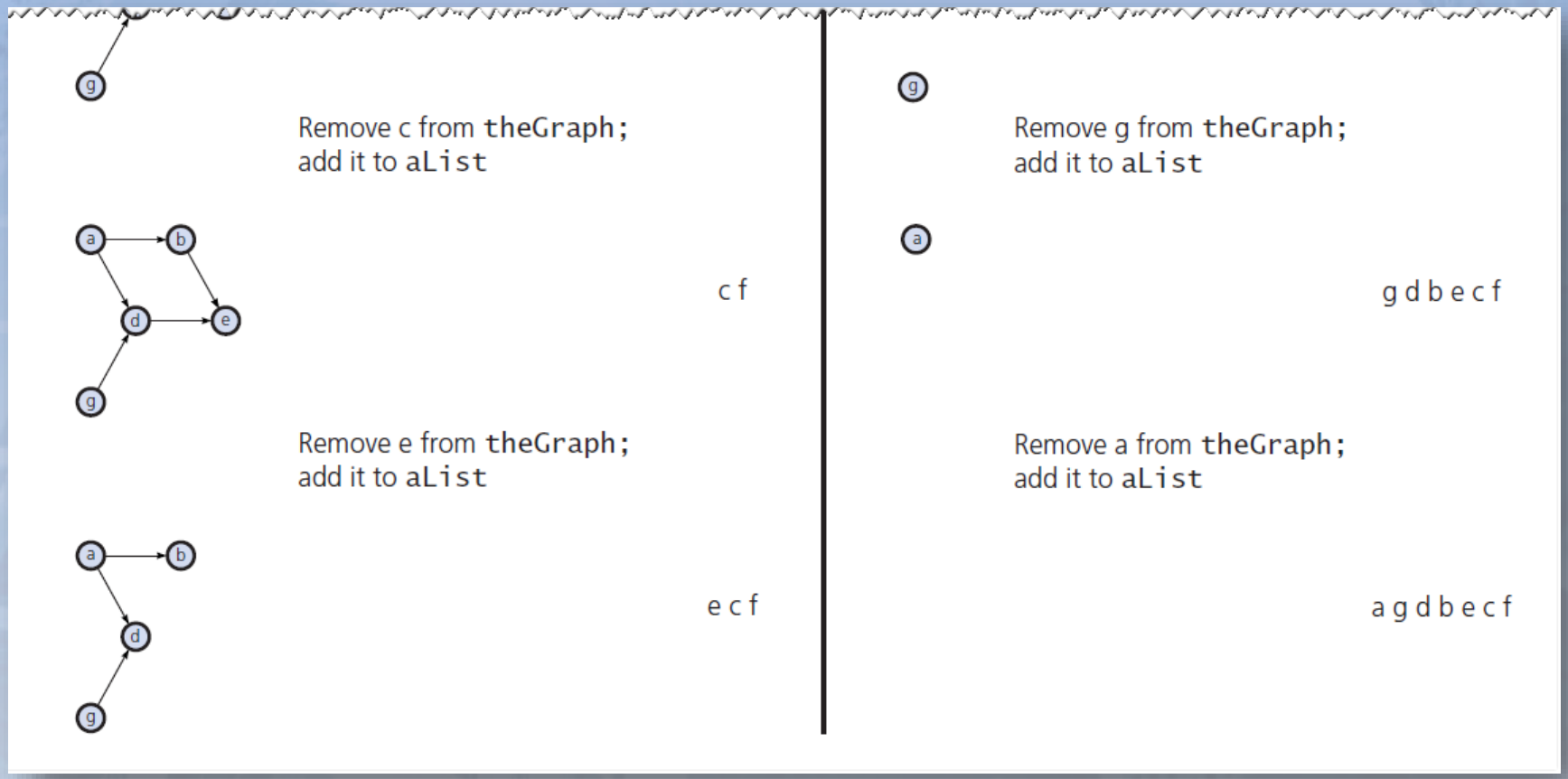


FIGURE 20-17 A trace of topSort1 for the graph in Figure 20-15

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Topological Sorting

Action	Stack s (bottom to top)	List aList (beginning to end)
Push a	a	
Push g	a g	
Push d	a g d	
Push e	a g d e	c
Push c	a g d e c	c
Pop c, add c to aList	a g d e	f c
Push f	a g d e f	e f c
Pop f, add f to aList	a g d e	d e f c
Pop e, add e to aList	a g d	g d e f c
Pop d, add d to aList	a g	g d e f c
Pop g, add g to aList	a	b g d e f c
Push b	a b	a b g d e f c
Pop b, add b to aList	a	
Pop a, add a to aList	(empty)	

FIGURE 20-18 A trace of topSort2 for the graph in Figure 20-15

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

??????????????

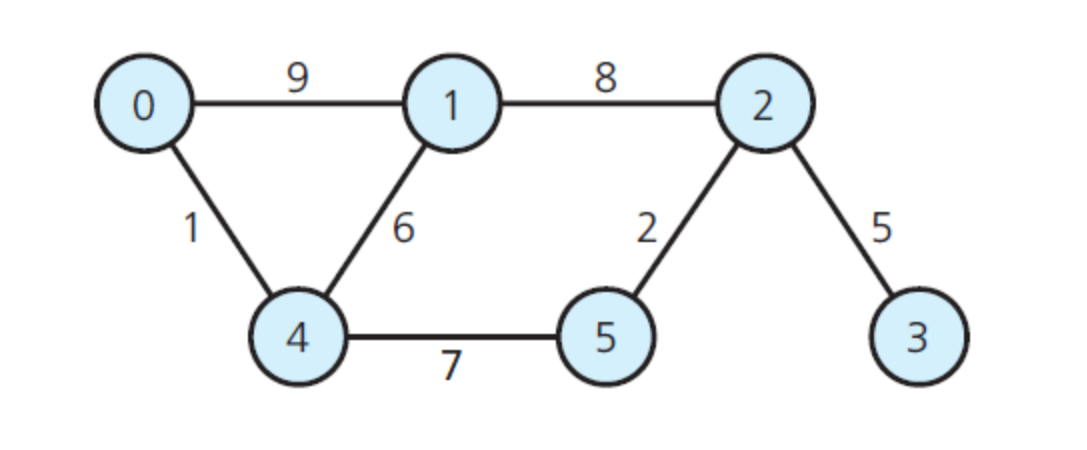


Figure 20-22 A graph for Checkpoint Questions 8, 9, and 10 and for Exercises 1 and 4

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits

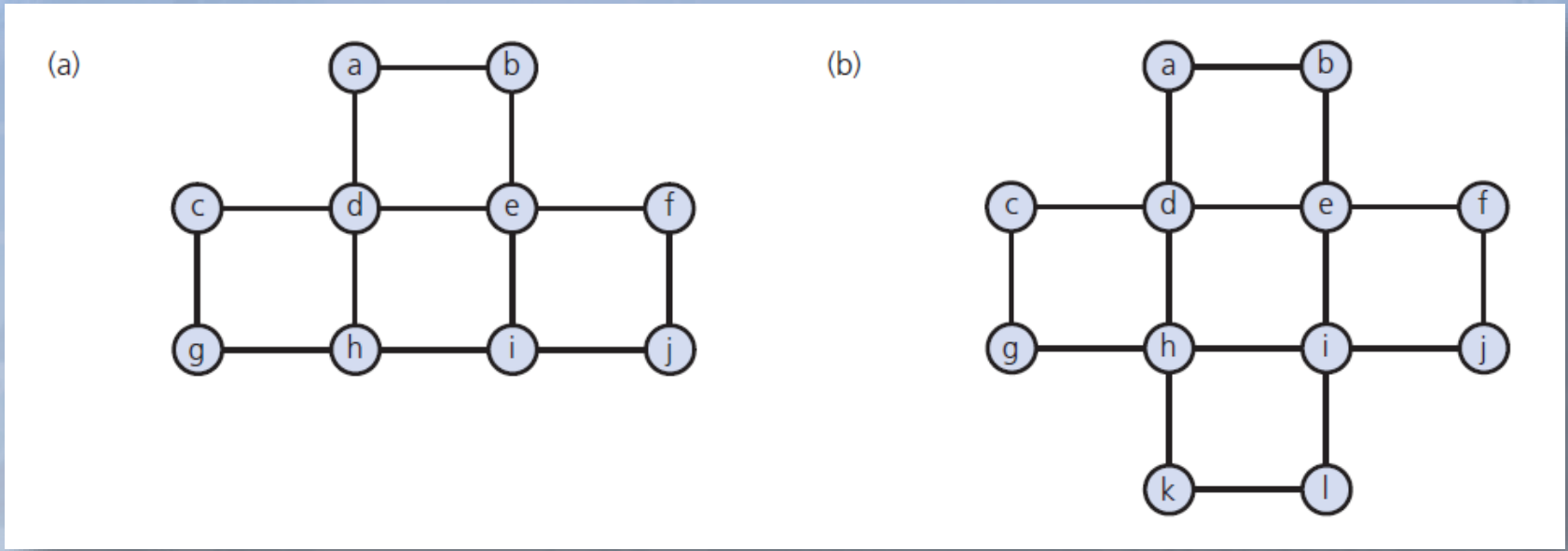


FIGURE 20-30 Connected undirected graphs based on the drawings in Figure 20-29

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits

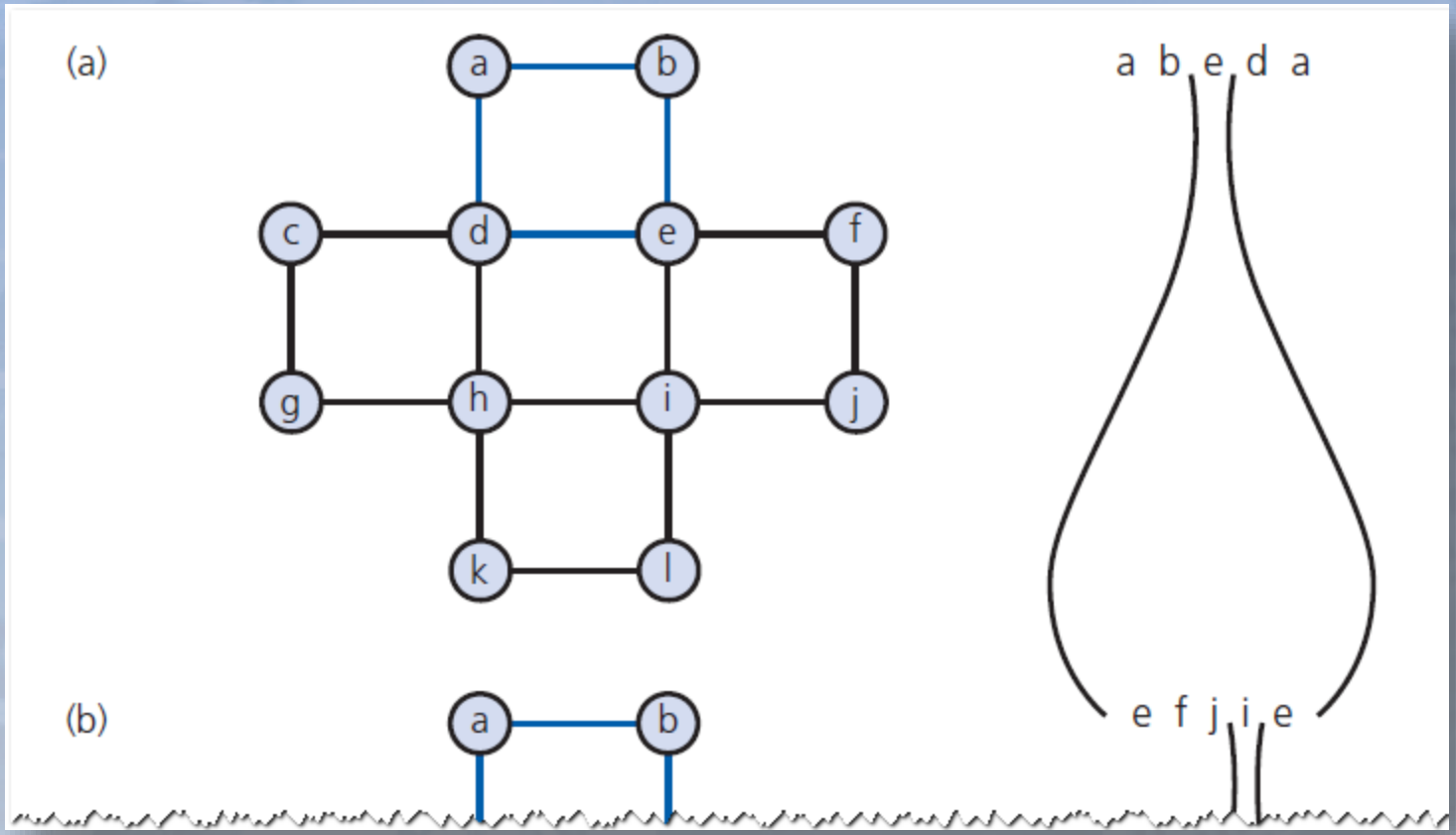


FIGURE 20-31 The steps to find an Euler circuit for the graph in Figure 20-30b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits

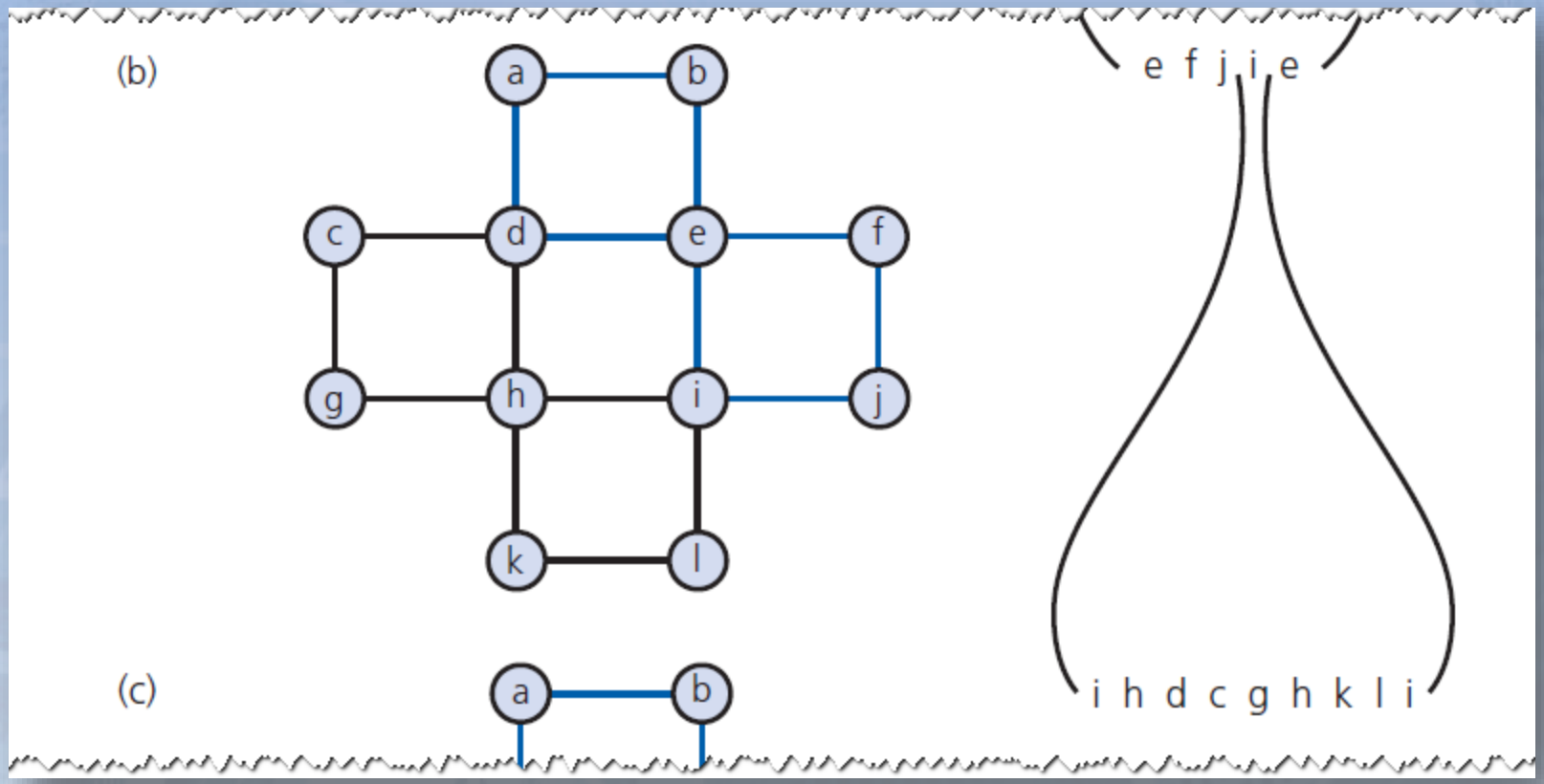


FIGURE 20-31 The steps to find an Euler circuit for the graph in Figure 20-30b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Circuits

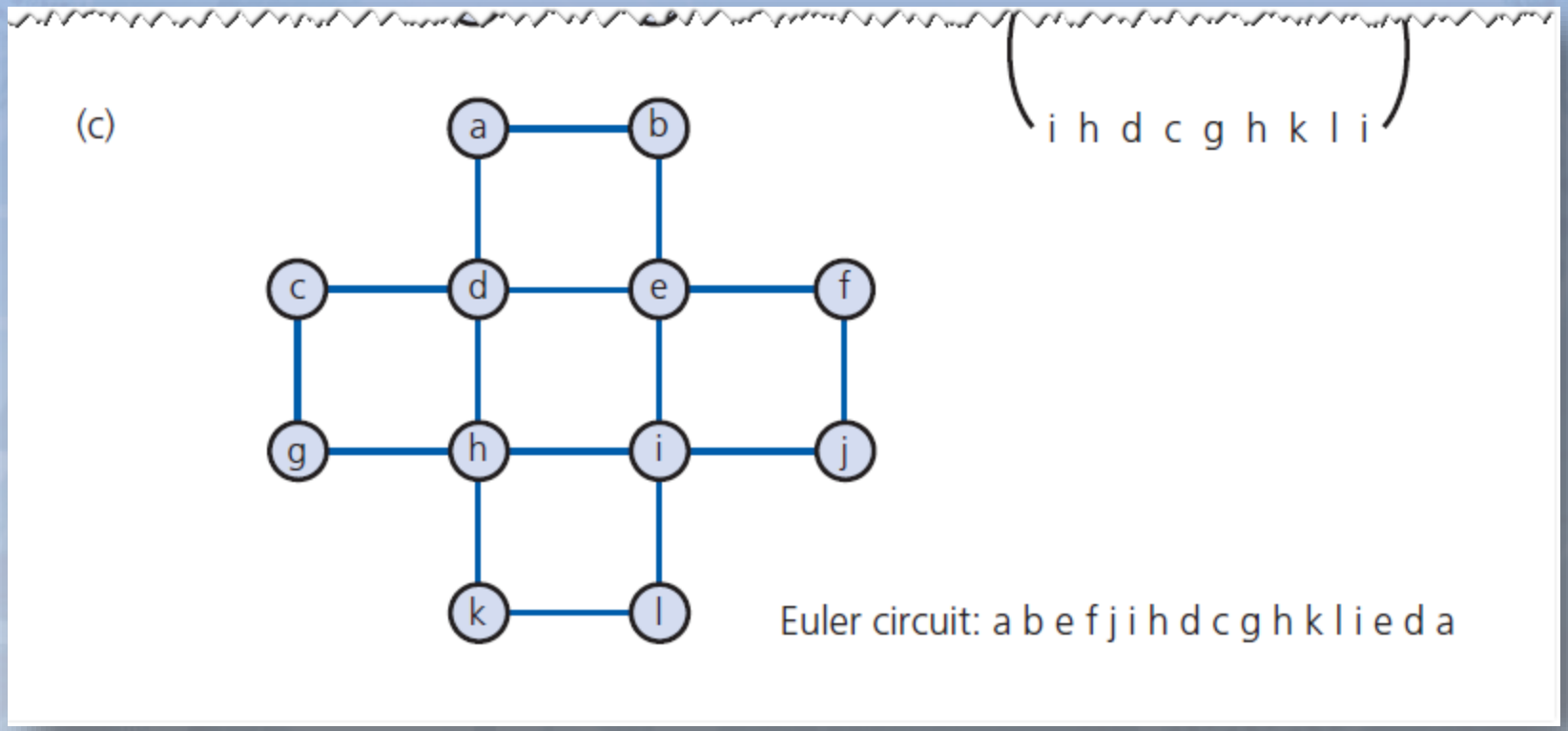


FIGURE 20-31 The steps to find an Euler circuit for the graph in Figure 20-30b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Some Difficult Problems

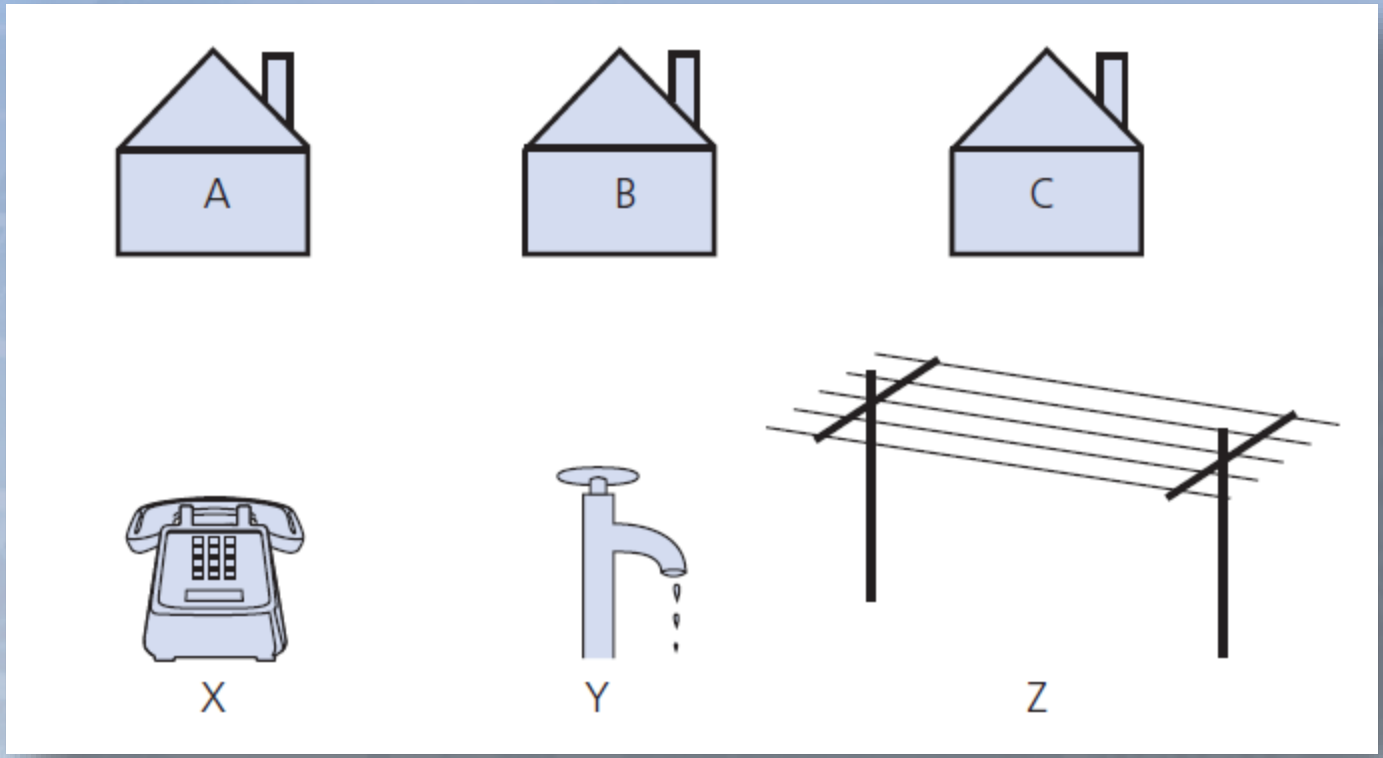


FIGURE 20-32 The three utilities problem

© 2017 Pearson Education, Hoboken, NJ. All rights reserved